

Bot de Trading

Samuel Hoyos R

Julio 2026

Resumen

Este sistema se basará para la inversión automática en mercados volátiles de criptomonedas bajo la plataforma **Trading View** con una cuenta demo tipo **FTMO**.

Para esto, se crea un ciclo con un filtro adaptativo de kalman extendido (*EAKF*), proseguido por un algoritmo de control predictivo basado en modelos no lineales (*nMPC*). Tomando como modelo de la matriz dinámica (A) el modelo *DS* ajustando la acción de control con el modelo *Minimax-Loeper*.

1. Canción del Micelio

Considere la acumulación de volumen neto $\sum Q$ durante un periodo donde el precio de un activo S regresa a un punto anterior $S_0 = S_t$. La inversa de los ticks necesarios para completar este ciclo es denotada como *frecuencia de mercado* ω_m . Esta frecuencia no mide tiempo físico continuo, sino periodos discretos de mercado (ticks).

A partir de estas variables, se estima el flujo de mercado Φ , que representa la tasa de inyección o extracción de capital por tick:

$$\Phi = \sum Q \cdot \omega_m$$

Este modelo compila múltiples frecuencias y sus respectivas acumulaciones de volumen neto, asumiendo que el mercado exhibe una tendencia oscilatoria errática con reversiones a ceros.^o niveles de referencia (nodos de precio).

Denotamos $\bar{T}$ como el rango continuo (o diferencial) de ticks donde convergen frecuencias de mercado similares. Tomamos el precio del activo S_0 al inicio de $\bar{T}$ y lo restamos del precio final S_f , definiendo el desplazamiento de precio $\Delta S = S_f - S_0$.

A partir de esta relación entre el flujo y el desplazamiento del precio, definimos la función del modelo de mercado Ψ :

$$\Psi = \frac{\Phi}{\Delta S}$$

Para analizar la dinámica del activo bajo este modelo, establecemos los siguientes estados en función de Φ y ΔS :

1. $(\Phi > \bar{\Phi}) \wedge (\Delta S < 0)$: Alta inyección de flujo, pero pérdida de valor (Absorción de demanda o Distribución).
2. $(\Phi > \bar{\Phi}) \wedge (\Delta S > 0)$: Alta inyección de flujo y ganancia de valor (Acumulación o Tendencia confirmada).
3. $\frac{\partial \Psi}{\partial T} = 0$: Significa que el mercado mantiene su estabilidad estructural (la relación flujo/desplazamiento es constante).
4. $(\Phi > \bar{\Phi}) \wedge (\Delta S \approx 0)$: Alto flujo de volumen sin desplazamiento de precio (Modelo Coca-Cola: alta liquidez, baja volatilidad direccional).
5. $(\Phi < \bar{\Phi}) \wedge (\Delta S > 0)$: Bajo flujo de volumen, pero el activo gana valor (Modelo Ferrari: exclusividad, poca liquidez necesaria para mover el precio al alza).

Para evaluar la vigencia de estos rangos, definimos la estabilidad del mercado Ω como la tasa de cambio de Ψ respecto al rango de ticks $\bar{T}$:

$$\Omega = \frac{\partial \Psi}{\partial \bar{T}}$$

1.1. Unidades Ajustadas (Consistencia Dimensional)

Para garantizar que la métrica de liquidez del Micelio se acople perfectamente a la Ecuación Diferencial Parcial de Minimax-Loeper y evite singularidades matemáticas en el cálculo de impacto, las unidades se definen estrictamente en función del par de negociación inverso (e.g., USD/BTC):

- **Acumulación de Volumen Neto** ($\sum Q$): USD.
- **Frecuencia de Mercado** (ω_m): $\frac{1}{\text{Ticks}}$.
- **Desplazamiento del Precio** (ΔS): $\frac{\text{USD}}{\text{BTC}}$.
- **Flujo de Mercado** ($\Phi = \sum Q \cdot \omega_m$): $\frac{\text{USD}}{\text{Ticks}}$.
- **Modelo de Mercado** ($\Psi = \frac{\Phi}{\Delta S}$): $\frac{\text{BTC}}{\text{Ticks}}$.
- **Estabilidad del Modelo** ($\Omega = \frac{\partial \Psi}{\partial \bar{T}}$): $\frac{\text{BTC}}{\text{Ticks}^2}$.

1.2. Interpretación Física

- **Acumulación de Volumen Neto ($\sum Q$):** Nos indica cuán activo financieramente ha estado el mercado y cuánta energía (dinero) ha interactuado en el sistema durante un ciclo.
- **Frecuencia de Mercado (ω_m):** Representa la rapidez con la que el mercado regresa a un nodo de precio de referencia, haciendo una analogía con la frecuencia de un oscilador resonante.
- **Desplazamiento del Precio (ΔS):** Determina el resultado direccional del activo (trabajo neto) al final de una secuencia de frecuencias.
- **Flujo de Mercado (Φ):** Determina la intensidad del mercado; cuánto dinero (energía) está fluyendo en el sistema por unidad de tick.
- **Modelo de Mercado (Ψ):** Análogo a una admitancia.^o inversa de resistencia. Nos dice qué tanto flujo de capital se requiere para generar una unidad de desplazamiento en el precio.
- **Estabilidad del Modelo (Ω):** Aceleración o cambio en la dinámica. Indica si la relación de esfuerzo (flujo) y resultado (precio) se está rompiendo (transición de fase del mercado) o se mantiene estable ($\Omega = 0$).

1.3. Visualización Gráfica

A través de la siguiente figura, podemos ver cómo se consideraría cada frecuencia y cada rango de tiempo:

Figura 1: Esquema de frecuencias de retorno y acumulación de volumen en el espacio de ticks.

1.4. Ecuación diferencial en el tiempo continuo

Si expandimos la definición de Ω sustituyendo Ψ , obtenemos la ecuación diferencial en función de las variables fundamentales. Partiendo de:

$$\Omega = \frac{\partial}{\partial \bar{T}} \left(\frac{\sum Q \cdot \omega_m}{\Delta S} \right)$$

Aplicamos la regla del cociente para la derivada parcial respecto a $\bar{T}$:

$$\Omega = \frac{\Delta S \cdot \frac{\partial(\sum Q \cdot \omega_m)}{\partial \bar{T}} - (\sum Q \cdot \omega_m) \cdot \frac{\partial \Delta S}{\partial \bar{T}}}{\Delta S^2}$$

Expandiendo la derivada del producto en el numerador ($\sum Q \cdot \omega_m$):

$$\Omega = \frac{\Delta S \left(\omega_m \frac{\partial \sum Q}{\partial T} + \sum Q \frac{\partial \omega_m}{\partial T} \right) - (\sum Q \cdot \omega_m) \frac{\partial \Delta S}{\partial T}}{\Delta S^2}$$

Separando en fracciones para facilitar su análisis numérico y aislar el efecto de la variación del precio:

$$\Omega = \frac{1}{\Delta S} \left(\omega_m \frac{\partial \sum Q}{\partial T} + \sum Q \frac{\partial \omega_m}{\partial T} \right) - \frac{\sum Q \cdot \omega_m}{\Delta S^2} \frac{\partial \Delta S}{\partial T}$$

Esta formulación final aísla los componentes de fuerza (variación de volumen y frecuencia) del componente de resistencia (variación del precio), permitiendo mapear la estabilidad computacionalmente.

2. Descomposición Empírica de Modos (EMD) en Sistemas de Control

La Transformada de Fourier clásica asume que las señales son estacionarias e infinitas, lo cual es ineficiente para entornos erráticos como los precios de mercado. Para resolver esto, utilizamos la Descomposición Empírica de Modos (EMD).

2.1. El Concepto Clásico y la Analogía del Barco

Físicamente, podemos imaginar el precio del mercado como la altura de un barco en el océano. La dinámica total está compuesta por:

- **Viento:** Ruido errático de alta frecuencia.
- **Oleaje:** Ciclos de frecuencia media.
- **Marea:** El offset móvil o tendencia base de baja frecuencia.

El algoritmo EMD utiliza un proceso iterativo llamado *Tamizado (Sifting)* que aísla los extremos locales (picos y valles) y calcula una línea media $m(\bar{T})$ mediante interpolación. Al restar $m(\bar{T})$ iterativamente hasta que la media sea cero, el algoritmo "pela" las capas de alta frecuencia, obteniendo Funciones de Modo Intrínseco (IMFs). El remanente final monótono es el residuo $R_n(\bar{T})$, que representa la marea u offset.

2.2. EMD de Ventana Deslizante (Tiempo Real)

El EMD clásico es un proceso *offline* que causa retardo temporal severo. Para inyectar el offset $R_n(t)$ como fuerza externa en la ecuación de estado de un Filtro de Kalman ($x_{n+1} = Ax_n + D \cdot R_n(\bar{T})$), implementamos una **Ventana Deslizante** de tamaño W que evalúa únicamente los datos recientes.

2.2.1. Ventana Adaptativa con Innovación de Kalman

Una ventana temporal de tamaño fijo es vulnerable a saltos abruptos o cambios de régimen macroeconómico. Para dotar al EMD de un comportamiento adaptativo, el tamaño de la ventana W se acopla a la innovación del Filtro de Kalman (el error de predicción del sistema).

Definimos la innovación estandarizada escalar ϵ_k en el instante k como:

$$\epsilon_k = \tilde{y}_k^T S_k^{-1} \tilde{y}_k$$

Donde $\tilde{y}_k$ es el vector de innovación teórica y S_k es su matriz de covarianza esperada. Un valor alto de ϵ_k indica una anomalía sistémica o un salto abrupto en el precio que el modelo físico no previó.

Para modelar la transición de la ventana temporal, empleamos un factor de decaimiento exponencial α :

$$\alpha = e^{-\gamma \epsilon_k}$$

Donde $\gamma > 0$ es un hiperparámetro de sensibilidad que calibra la reactividad del sistema ante la anomalía.

- Si $\epsilon_k \rightarrow 0$ (mercado estable), $\alpha \rightarrow 1$.
- Si $\epsilon_k \gg 0$ (salto abrupto), $\alpha \rightarrow 0$.

Finalmente, para asegurar que la ventana de datos no colapse a cero y mantenga coherencia en el indexado computacional (obligatoriamente números enteros de ticks), la ecuación que define el tamaño dinámico de la ventana W_k emplea una operación de redondeo al entero más cercano ($\lfloor \cdot \rfloor$):

$$W_k = \lfloor W_{min} + \alpha(W_{max} - W_{min}) \rfloor$$

Bajo esta formulación:

- Cuando el mercado es estable ($\alpha \approx 1$), W_k se expande asintóticamente hacia W_{max} , aumentando la resolución del EMD para detectar ciclos de baja frecuencia.
- Cuando ocurre una divergencia severa ($\alpha \approx 0$), W_k se contrae instantáneamente a W_{min} , descartando la memoria pasada para evitar que el *offset* histórico y caduco corrompa la estimación del nuevo régimen de precios.

2.3. Mitigación del Efecto de Borde (End-Effect) en la EMD

El cálculo en tiempo real de la Descomposición Empírica de Modos (EMD) adolece de una divergencia matemática conocida como el Efecto de Borde. Las envolventes superior e inferior, construidas mediante *splines* cúbicos, carecen de

extremos locales futuros para anclarse en el límite derecho del dominio temporal (el instante presente t_0).

Si la Transformada de Hilbert se aplica sobre Funciones de Modo Intrínseco (IMFs) afectadas por esta divergencia de borde, el cálculo de la fase instantánea $\theta(t)$ y la frecuencia estructural ω_m resultará en valores altamente erráticos en el último *tick*. Esto inyectaría ruido artificial en la derivada de estabilidad Ω , induciendo al optimizador NMPC a falsos bloqueos de protección.

Extensión de Señal por Reflexión y Modelado AR

Para preservar la integridad topológica del cálculo en el presente, se implementa una subrutina de extensión de señal (*Data Padding*) previa al proceso de tamizado (*sifting*) de la EMD.

Dado el vector histórico de precios en la ventana del Hilo Lento $S[t - W_k : t_0]$, se genera una extensión sintética de longitud M hacia un pseudo-futuro:

$$S_{ext} = [S_{t-W_k}, \dots, S_{t_0-1}, S_{t_0}, \hat{S}_{t_0+1}, \dots, \hat{S}_{t_0+M}]$$

Los valores sintéticos $\hat{S}$ se calculan utilizando una combinación de Reflexión Simétrica de los últimos extremos y una predicción por Modelo Autoregresivo (AR) de bajo orden, asegurando que la primera y segunda derivada de la unión sean continuas.

El flujo algorítmico se corrige de la siguiente forma:

1. Se extiende la serie temporal S a S_{ext} .
2. Se ejecuta la EMD y la Transformada de Hilbert sobre el vector completo S_{ext} .
3. Se truncan los tensores resultantes, descartando los M puntos finales, extrayendo las variables analíticas exactas para el instante verdadero t_0 .

Esta corrección matemática fuerza a que el “aleteo” del *spline* ocurra en el dominio sintético descartado, entregando al Filtro de Kalman y al controlador NMPC métricas de fase (θ, Ω) estrictamente puras y libres de distorsión de contorno.

2.4. Análisis de Frecuencia: Transformada de Hilbert y Desenrollado de Fase

Para obtener la frecuencia exacta de una oscilación en un instante específico sin depender de ventanas temporales infinitas (como en Fourier), utilizamos la **Transformada de Hilbert**.

2.4.1. La Señal Analítica y la Fase

Dada una oscilación unidimensional en el tiempo (como el oleaje), la Transformada de Hilbert crea una "sombra imaginaria desfasada 90°". Esto convierte la señal real $X(\bar{T})$ en una señal analítica en el plano complejo:

$$Z(\bar{T}) = A(\bar{T})e^{i\theta(\bar{T})}$$

Donde $A(\bar{T})$ es la amplitud instantánea y $\theta(\bar{T})$ es el ángulo de fase.

2.4.2. El Problema del Salto y el Desenrollado de Fase (Phase Unwrapping)

El cálculo de $\theta(\bar{T})$ mediante funciones trigonométricas (ej. arcotangente) devuelve valores acotados (típicamente entre $-\pi$ y π). Cuando el ciclo se completa, la fase sufre un reinicio violento, saltando de π a $-\pi$. Si derivamos este salto, obtenemos una singularidad matemática (una frecuencia negativa infinita).

Para solucionar esto, aplicamos un **Desenrollado de Fase**. Este algoritmo detecta los saltos mayores a π y suma acumulativamente 2π a todos los valores sucesivos. Esto transforma la fase acotada en una función monótona creciente continua (fase acumulada).

2.4.3. Frecuencia Instantánea

Con la fase desenrollada $\theta_{unwrap}(\bar{T})$, la frecuencia instantánea $f(\bar{T})$ se calcula derivando analítica o numéricamente la curva respecto al tiempo:

$$f(\bar{T}) = \frac{1}{2\pi} \frac{d\theta_{unwrap}(\bar{T})}{d\bar{T}}$$

2.4.4. La Arquitectura Completa: Transformada de Hilbert-Huang (HHT)

La combinación del EMD (Descomposición Empírica de Modos) y la Transformada de Hilbert conforma el método HHT, ideal para señales no lineales y no estacionarias como los precios de mercado. El flujo de trabajo se ensambla de la siguiente manera:

1. **Tamizado (EMD):** Se descompone la señal errática original en múltiples Funciones de Modo Intrínseco (IMFs) y un residuo $R_n(t)$.
2. **Aislamiento de Dinámicas:**
 - Las primeras IMFs representan el ruido de alta frecuencia (volatilidad).
 - Las IMFs intermedias representan los ciclos dominantes del sistema.
 - El residuo $R_n(t)$ aísla la tendencia no lineal (offset dinámico).

3. **Espectro de Hilbert:** A cada IMF individual se le aplica la Transformada de Hilbert y el desenrollado de fase. Esto nos permite graficar un espectro tridimensional (Tiempo, Frecuencia Instantánea y Amplitud) con alta resolución, logrando identificar exactamente cuándo y con qué intensidad cambia la frecuencia dominante del mercado.

2.5. Colapso Espectral: Frecuencia Instantánea Ponderada

El Espectro de Hilbert genera una matriz tridimensional donde cada Función de Modo Intrínseco (IMF) posee su propia frecuencia y amplitud simultánea. Dado que el modelo de mercado (la Canción del Micelio) requiere un escalar único ω_m para acoplarse al impacto de Loeper (λ), es matemáticamente incorrecto sumar las frecuencias de cada IMF de forma directa, ya que esto violaría el principio de superposición no lineal.

Para resolver esta indeterminación, se extrae el centroide energético del sistema. La premisa física establece que la verdadera dinámica del mercado en un instante discreto $\bar{T}$ está dictada por el ciclo temporal que moviliza la mayor cantidad de capital, lo cual se ve reflejado en la amplitud de la onda.

A partir de la señal analítica generada por la Transformada de Hilbert, cada modo i posee una amplitud instantánea $A_i(\bar{T})$ y una frecuencia instantánea evaluada estrictamente en el dominio de los ticks:

$$f_i(\bar{T}) = \frac{1}{2\pi} \frac{d\theta_{i,unwrap}}{d\bar{T}}$$

Se define la energía instantánea de cada modo estructural como el cuadrado de su amplitud, $E_i(\bar{T}) = A_i^2(\bar{T})$.

Para evitar inestabilidades en el control predictivo, se filtran los extremos del espectro espectral: se descarta sistemáticamente la IMF_1 (cuyo comportamiento se asocia casi enteramente al ruido blanco de alta frecuencia del *bid-ask spread*) y el residuo R_n (que representa la tendencia asintótica carente de fase cíclica).

La frecuencia de mercado escalar consolidada ω_m se calcula como el promedio de las frecuencias estructurales ponderado por su energía instantánea:

$$\omega_m(\bar{T}) = \frac{\sum_{i=2}^{n-1} A_i^2(\bar{T}) \cdot f_i(\bar{T})}{\sum_{i=2}^{n-1} A_i^2(\bar{T})}$$

Esta formulación matemática garantiza que si un flujo institucional altera bruscamente una escala específica del precio, su respectiva IMF ganará amplitud. Al estar elevada al cuadrado, esta energía arrastrará el valor de ω_m hacia el ciclo dominante, forzando a la matriz de covarianza adaptativa Q del filtro de Kalman y al presupuesto Minimax a reaccionar de forma endógena y sin latencia al nuevo régimen de volatilidad.

2.6. Redefinición de la Referencia de Precio: Nodos de Fase (HHT)

Para que el cálculo del desplazamiento de precio ΔS sea dinámico y conserve el sentido físico establecido en la Canción del Micelio (donde el precio regresa a un punto anterior de equilibrio), se abandona el uso de un precio histórico arbitrario. En su lugar, se emplea la señal analítica de la Transformada de Hilbert para detectar los momentos exactos en los que el mercado completa un medio ciclo estructural.

Definimos un “nodo de fase” como el instante discreto en ticks $\bar{T}_{nodo}$ en el cual la fase desenrollada de la dinámica dominante cruza el eje horizontal (amplitud cero). Matemáticamente, este evento de reversión se detecta cuando la fase cumple la condición:

$$\theta_{unwrap}(\bar{T}_{nodo}) \equiv 0 \pmod{\pi}$$

En cada cruce detectado, el sistema captura el precio absoluto del activo, actualizando la referencia dinámica del ciclo vigente:

$$S_{ref} = S(\bar{T}_{nodo})$$

Por lo tanto, el desplazamiento de precio en cualquier instante actual $\bar{T}$ se define como la diferencia entre el precio presente y el precio anclado en el último nodo estructural completado:

$$\Delta S(\bar{T}) = S(\bar{T}) - S_{ref}$$

Interpretación y Ventaja del Modelo

Bajo este paradigma, el algoritmo no mide el desplazamiento contra el infinito histórico, sino que evalúa el esfuerzo direccional del mercado estrictamente dentro de su respiración cíclica actual.

- Un escenario con $\Delta S > 0$ indica que la inyección de liquidez neta (Φ) está logrando desplazar el activo positivamente respecto a su último punto de equilibrio.
- Un escenario con $\Delta S < 0$ señala debilidad estructural; a pesar de la energía ingresada al sistema, el activo cotiza por debajo de su nodo de referencia (estado de distribución).

El retorno del precio exactamente al nodo ($S(\bar{T}) \rightarrow S_{ref}$) fuerza a $\Delta S \rightarrow 0$. Esto dispara el valor de Ψ y, consecuentemente, eleva la falta de liquidez λ en el modelo de Loeper, alertando matemáticamente al controlador NMPC de un punto de alta tensión estructural donde la ejecución de órdenes generará un impacto de mercado masivo.

3. Filtro de Kalman

3.1. Introducción al Filtro de Kalman (Enfoque Escalar)

El Filtro de Kalman es un algoritmo recursivo que estima el estado real de un sistema dinámico minimizando la varianza del error. Funciona en un ciclo de dos pasos: **Predicción** (modelo físico) y **Corrección** (medición del sensor).

3.1.1. El Horno de Temperatura Estable

Asumiendo un horno que debe mantenerse a 500°C con un sensor ruidoso. Como solo hay una variable (temperatura x) y no hay dinámica de cambio, el sistema es escalar.

■ **Predicción:**

$$x_{pred} = x_{ant}$$

$$p_{pred} = p_{ant} + q$$

■ **Ganancia de Kalman (K):**

$$K = \frac{p_{pred}}{p_{pred} + r}$$

■ **Corrección:**

$$x_{nuevo} = x_{pred} + K(z - x_{pred})$$

$$p_{nuevo} = (1 - K)p_{pred}$$

Donde r es el ruido del sensor, q el ruido del proceso y z la lectura real de la termocupla.

3.2. El Filtro Lineal Matricial

Cuando el sistema tiene múltiples variables acopladas (ej. posición y velocidad) o cambia en el tiempo (el horno calentándose), pasamos al formato matricial.

3.2.1. El Coche en el Túnel y Pérdida de Señal

Consideramos un coche en movimiento. El estado incluye posición (p) y velocidad (v).

$$x = \begin{bmatrix} p \\ v \end{bmatrix}$$

La cinemática define la matriz de transición de estado (A):

$$x_{pred} = Ax_{ant} \implies \begin{bmatrix} p_{pred} \\ v_{pred} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{ant} \\ v_{ant} \end{bmatrix}$$

El GPS solo mide la posición, definiendo la matriz de observación (H):

$$z_{esperado} = Hx_{pred} \implies z = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p \\ v \end{bmatrix}$$

En el túnel (Sin señal GPS): Se omite la fase de corrección. La incertidumbre de la predicción (P) crece iteración tras iteración:

$$P_{pred} = AP_{ant}A^T + Q$$

Al salir del túnel, la Ganancia de Kalman (K) será alta para la posición, confiando fuertemente en la nueva lectura del GPS.

3.2.2. Ruido del Proceso (Q) - El Aeropéndulo

Las perturbaciones externas, como ráfagas de viento sobre un aeropéndulo, se traducen en una aceleración aleatoria con varianza σ_a^2 . Su propagación en la posición y velocidad genera la matriz de covarianza Q :

$$Q = \sigma_a^2 \begin{bmatrix} \frac{1}{4}\Delta t^4 & \frac{1}{2}\Delta t^3 \\ \frac{1}{2}\Delta t^3 & \Delta t^2 \end{bmatrix}$$

Los términos cruzados indican cómo el ruido acopla geoméricamente el error de las variables.

3.3. La Innovación

La innovación ($\tilde{y}_k$) representa la discrepancia entre la medición real y la predicción del modelo:

$$\tilde{y}_k = z_k - Hx_{pred}$$

Su covarianza teórica esperada (S_k) se define como:

$$S_k = HP_{pred}H^T + R$$

Esto permite reescribir la actualización del estado como:

$$x_{nuevo} = x_{pred} + K_k\tilde{y}_k$$

3.4. Filtro de Kalman Extendido (EKF)

Para sistemas no lineales, las matrices A y H no existen de forma constante. El EKF linealiza el sistema calculando la Matriz Jacobiana en cada instante de tiempo (la "baldosa plana").

3.4.1. El Péndulo Trigonométrico

Si un sensor óptico mide la distancia horizontal (cateto opuesto) del brazo del péndulo de longitud L , la función de medición no lineal es $h(\theta, \omega) = L \sin(\theta)$.

El Jacobiano de observación (J_H) se obtiene derivando parcialmente respecto al estado:

$$J_H = \begin{bmatrix} \frac{\partial h}{\partial \theta} & \frac{\partial h}{\partial \omega} \end{bmatrix} = \begin{bmatrix} L \cos(\theta) & 0 \end{bmatrix}$$

Cuando el péndulo está vertical ($\theta = 0^\circ$), $J_H = [L, 0]$, indicando máxima sensibilidad del sensor. Cuando está horizontal ($\theta = 90^\circ$), $J_H = [0, 0]$, el sensor pierde utilidad matemática.

3.5. Filtro de Kalman Adaptativo (AKF)

El AKF ajusta la matriz de ruido del proceso (Q) dinámicamente cuando el sistema sufre perturbaciones no previstas en el modelo físico original.

Mediante *Covariance Matching*, se compara la covarianza teórica (S_k) con la covarianza estadística real de la innovación ($\hat{C}_k$), calculada en una ventana de las últimas N mediciones:

$$\hat{C}_k = \frac{1}{N} \sum_{i=k-N+1}^k \tilde{y}_i \tilde{y}_i^T$$

Si $\hat{C}_k \gg S_k$, el modelo está divergiendo. La nueva matriz adaptativa $\hat{Q}$ se recalcula como el exceso de caos real frente a la certidumbre teórica:

$$H\hat{Q}H^T = \hat{C}_k - H(AP_{ant}A^T)H^T - R$$

Esto expande Q dinámicamente, abriendo la ganancia para confiar más en los sensores hasta que el evento transitorio cese.

3.6. Aumento de Estado (Estimación de Parámetros Ocul-tos)

Esta técnica permite expandir las dimensiones de las matrices del filtro para estimar variables físicas desconocidas o defectos sistemáticos del hardware (como fricciones no modeladas o sesgos constantes) en tiempo real. Es crucial diferenciar esto del ruido de proceso (Q) o medición (R), que manejan variaciones aleatorias de media cero; el aumento de estado se utiliza específicamente para lidiar con errores direccionales, persistentes y constantes.

3.6.1. Ejemplo Práctico: La Báscula Descalibrada

Supongamos que medimos una variable física como el peso (p) y su tasa de cambio o velocidad (v), pero sabemos que el sensor (la báscula) tiene un sesgo o desvío de fábrica constante (b). En lugar de que el filtro se confunda e intente

asignar este error continuo a fluctuaciones físicas, convertimos a b en una nueva incógnita dinámica a resolver.

El vector de estado original se .aumenta.añadiendo esta nueva variable en la última posición:

$$x = \begin{bmatrix} p \\ v \\ b \end{bmatrix}$$

1. Expansión de la Matriz de Transición (A): Dado que el sesgo es un defecto de hardware constante en el tiempo, su dinámica es estática ($b_{pred} = b_{ant}$). La matriz del modelo físico se expande añadiendo una fila y una columna que conecta a b únicamente consigo mismo, sin verse afectado por p o v :

$$\begin{bmatrix} p_{pred} \\ v_{pred} \\ b_{pred} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_{ant} \\ v_{ant} \\ b_{ant} \end{bmatrix}$$

2. Expansión de la Matriz de Observación (H): La lectura del sensor (z) no entrega el peso puro, sino una lectura viciada que es la suma del peso más el sesgo ($z = p + b$). Para que el álgebra matricial extraiga exactamente esta combinación del vector de estado x , la matriz H se diseña .encendiendo.^a p y b , y .apagando.^a v :

$$H = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

3. La Mecánica de Convergencia: Para que el filtro descubra el sesgo, se debe inicializar la matriz de covarianza (P) asignándole una gran incertidumbre inicial al parámetro b (asumiendo que dudamos de la calibración de fábrica). Ciclo tras ciclo, el sensor arrojará un error persistente. Como las variables físicas (p y v) están rígidamente atadas a las leyes del movimiento en A , el filtro detectará la contradicción usando los términos cruzados de la matriz P . La matemática deducirá que la única explicación lógica para un error constante es el sesgo. Así, la componente respectiva de la Ganancia de Kalman (K_b) absorberá el error, aislando la incógnita b (por ejemplo, 2 kg) y permitiendo al sistema restar este valor para conocer el peso real.

4. Modelo MiniMax-Loeper

4.1. Introducción: El Problema de la Cobertura

Para comprender este modelo, primero debemos entender qué hace un creador de mercado (market maker). Cuando un operador vende una opción financiera (un contrato que da el derecho a comprar o vender un activo en el futuro), asume un riesgo inmenso si el precio del activo subyacente (S) se mueve en su contra.

Para anular este riesgo, el operador realiza una **cobertura dinámica**. Compra o vende una cantidad exacta del activo subyacente para que las ganancias en el activo compensen las pérdidas de la opción. Esta cantidad se conoce como **Delta** (Δ).

$$\Delta = \frac{\partial U}{\partial S}$$

Donde U es el precio de la opción y S es el precio del activo.

El modelo tradicional (Black-Scholes) asume dos cosas que son falsas en mercados como el de las criptomonedas:

1. Que el precio se mueve como una onda probabilística predecible (Movimiento Browniano).
2. Que el mercado tiene liquidez infinita (comprar o vender no cambia el precio).

Minimax y Loeper solucionan estos dos errores respectivamente.

4.2. El Modelo Minimax: El Mercado como Adversario

En lugar de usar probabilidades estocásticas, el enfoque **Minimax** imagina el mercado como un juego de suma cero. No hay “ruido aleatorio”; hay un **Adversario** (el mercado) que elige intencionalmente la trayectoria del precio S para causar el mayor daño financiero (arrepentimiento o *regret*) al operador.

La única restricción que tiene el Adversario es un “presupuesto de volatilidad” máximo, denotado como c_{vol}^2 . Matemáticamente, si el operador juega óptimamente para minimizar su peor escenario posible, la Ecuación Diferencial Parcial (EDP) que rige el valor de la opción es:

$$\frac{\partial U}{\partial t} + \frac{1}{2}c_{vol}^2 S^2 \frac{\partial^2 U}{\partial S^2} = 0 \quad (1)$$

Aquí, c_{vol}^2 reemplaza a la varianza estocástica clásica (σ^2). Sin embargo, esto aún asume liquidez infinita.

4.3. El Impacto de Loeper: La Paradoja de la Cobertura

Grégoire Loeper revolucionó este campo al demostrar que la acción de cubrirse altera el precio del activo. Esta es la deducción lógica, paso a paso:

4.3.1. La Necesidad de Rebalanceo (Gamma)

Cuando el precio del mercado cambia una cantidad infinitesimal dS , el Delta del operador cambia. La tasa de cambio de Delta se llama **Gamma** (Γ):

$$\Gamma = \frac{\partial \Delta}{\partial S} = \frac{\partial^2 U}{\partial S^2}$$

Para seguir cubierto, el operador está **obligado** a ir al mercado y transar un volumen V equivalente a:

$$V = \Gamma \cdot dS \quad (2)$$

4.3.2. El Desplazamiento del Precio (Impacto de Mercado)

En la realidad, toda orden empuja el precio. Loeper modela que lanzar un volumen V desplaza el precio en una proporción dependiente de un parámetro de falta de liquidez λ y el precio al cuadrado:

$$\text{Impacto del Trader} = \lambda S^2 \cdot V \quad (3)$$

4.3.3. El Bucle de Retroalimentación

Si sustituimos el volumen V que el trader está obligado a operar, el impacto que él mismo genera es $\lambda S^2(\Gamma \cdot dS)$. Por lo tanto, el movimiento **total** del mercado (dS_{total}) es la suma de un choque externo (dS_{ext}) más el impacto endógeno del trader:

$$dS_{total} = dS_{ext} + \lambda S^2 \Gamma \cdot dS_{total}$$

Factorizando dS_{total} :

$$dS_{total}(1 - \lambda S^2 \Gamma) = dS_{ext}$$

$$dS_{total} = \frac{dS_{ext}}{1 - \lambda S^2 \Gamma}$$

Dado que la volatilidad (varianza) es proporcional al cuadrado de los movimientos de precio, la fuerza destructiva del mercado (c_{vol}^2) se ve multiplicada por este factor estructural.

4.4. La Ecuación Unificada Minimax-Loeper

Acoplando la varianza efectiva de Loeper en la dinámica adversarial de Minimax (y añadiendo los componentes de descuento financiero r_{USD} y q_{BTC}), obtenemos la EDP final:

$$\frac{\partial U}{\partial t} + \frac{1}{2} \left[\frac{c_{vol}^2 S^2}{1 - \lambda S^2 \frac{\partial^2 U}{\partial S^2}} \right] \frac{\partial^2 U}{\partial S^2} + (r_{USD} - q_{BTC}) S \frac{\partial U}{\partial S} - q_{BTC} U = 0 \quad (4)$$

4.4.1. Análisis Dimensional

Para garantizar la coherencia física de la ecuación (especialmente modelando opciones inversas en criptomonedas, donde el contrato se liquida en el criptoactivo nativo, e.g., BTC), comprobamos las unidades:

- **Tiempo (t):** Años.
- **Precio (S):** USD / BTC.

- **Valor de la Opción (U):** BTC.
- **Delta ($\Delta = \frac{\partial U}{\partial S}$):** $\frac{\text{BTC}}{\text{USD/BTC}} = \frac{\text{BTC}^2}{\text{USD}}$.
- **Gamma ($\Gamma = \frac{\partial^2 U}{\partial S^2}$):** $\frac{\text{BTC}^2/\text{USD}}{\text{USD/BTC}} = \frac{\text{BTC}^3}{\text{USD}^2}$.
- **Presupuesto Minimax (c_{vol}):** $1/\sqrt{\text{Años}}$ (Varianza en Años^{-1}).

Análisis del Denominador de Loeper: Para que el término $1 - \lambda S^2 \Gamma$ pueda existir matemáticamente, $\lambda S^2 \Gamma$ debe ser adimensional.

$$[\lambda] \cdot [S^2] \cdot [\Gamma] = [\lambda] \cdot \left(\frac{\text{USD}}{\text{BTC}} \right)^2 \cdot \left(\frac{\text{BTC}^3}{\text{USD}^2} \right) = [\lambda] \cdot \text{BTC}$$

Por ende, las unidades obligatorias de la falta de liquidez son $[\lambda] = 1/\text{BTC}$. Representa el impacto en el mercado por cada unidad de criptoactivo transada.

4.4.2. La Singularidad de Loeper: Cuando Operar es la Ruina

El aspecto más crítico de este modelo radica en el denominador de la varianza efectiva:

$$\text{Denominador} = 1 - \lambda S^2 \Gamma \quad (5)$$

Matemáticamente, si la convexidad de la cartera (Γ) es muy alta y la liquidez es escasa (λ alto), el producto $\lambda S^2 \Gamma$ se aproxima a 1. Cuando esto ocurre, el denominador tiende a cero y la Ecuación Diferencial Parcial **explota** (tiende al infinito).

A este fenómeno se le conoce como la **Singularidad de Loeper**.

4.4.3. ¿Qué significa financieramente la explosión?

La singularidad revela una verdad física del mercado: **la ruptura absoluta del bucle de cobertura**.

Si el denominador cruza el cero (volviéndose negativo), la ecuación pasa de ser parabólica (difusiva) a un estado degenerado donde el problema está *mal planteado* de origen. En la práctica, significa que el trader es **demasiado grande para la piscina de liquidez**. Cada intento de comprar subyacente para ajustar su Delta mueve el precio tan abruptamente en su contra, que genera una nueva necesidad de cobertura aún mayor que la original. Es un ciclo de retroalimentación infinita e instantánea que aniquila el capital en milisegundos.

Conclusión Estratégica: Cuando el solver numérico detecta que $1 - \lambda S^2 \Gamma \leq \epsilon$ (donde ϵ es un margen de seguridad minúsculo), la recomendación matemática no es transar más para estabilizar la cartera. La directriz matemática es el **cierre forzoso o la inacción total en ese libro de órdenes**. Intentar rebalancear el portafolio en la vecindad de la singularidad no es una estrategia de riesgo; es un mecanismo matemático garantizado de autodestrucción.

4.5. Transformación del Modelo Minimax-Loeper: Tiempo Dual

Para evitar la "paradoja del mercado congelado"—donde una pausa transitoria en el volumen haría tender a cero el avance temporal y detonaría al infinito los términos de descuento financiero—, el modelo no transiciona de forma absoluta al tiempo de mercado. En su lugar, se emplea una *bifurcación del reloj del sistema*.

Definimos la tasa dinámica de llegada de ticks ν como la derivada del tiempo de mercado $\bar{T}$ respecto al tiempo físico cronológico t :

$$\nu = \frac{\partial \bar{T}}{\partial t} \quad \left[\frac{\text{Ticks}}{\text{Años}} \right]$$

Aplicando la regla de la cadena a la EDP unificada (Ecuación 4), separamos la difusión estocástica (gobernada exclusivamente por el volumen y la ejecución de órdenes) del decaimiento temporal (gobernado por el calendario continuo continuo).

La ecuación de Minimax-Loeper adaptada para operar de forma asíncrona en el algoritmo NMPC se reescribe como:

$$\underbrace{\nu \frac{\partial U}{\partial \bar{T}}}_{\text{Paso por Tick}} + \underbrace{\frac{\partial U}{\partial t}}_{\text{Decaimiento (Theta)}} + \underbrace{\frac{1}{2} \left[\frac{c_{vol}^2 S^2}{1 - \lambda S^2 \frac{\partial^2 U}{\partial S^2}} \right] \frac{\partial^2 U}{\partial S^2}}_{\text{Difusión Adversarial (Loeper)}} + \underbrace{(r_{USD} - q_{BTC}) S \frac{\partial U}{\partial S} - q_{BTC} U}_{\text{Descuento Financiero}} = 0$$

Bajo esta arquitectura dual, los componentes se resuelven iterativamente en dominios separados:

1. **Dominio Discreto ($\bar{T}$):** El término difusivo evalúa la convexidad de la cartera ($\Gamma = \frac{\partial^2 U}{\partial S^2}$) interactuando estrictamente con la llegada de liquidez. La falta de liquidez se actualiza dinámicamente desde el hilo lento como $\lambda = \frac{\omega_m}{\Psi}$.
2. **Acoplamiento de Volatilidad:** El presupuesto adversarial Minimax se acopla a la inestabilidad medida por el Micelio mediante la relación $c_{vol}^2 = k \cdot \omega_m \cdot \nu$, garantizando consistencia dimensional (1/Años).
3. **Dominio Continuo (t):** Los componentes independientes de costo del dinero (r_{USD} y q_{BTC}) y la propia pérdida de valor temporal de la opción se integran utilizando el diferencial de tiempo cronológico (Δt). Esto garantiza la continuidad matemática y protege el solver numérico incluso si la tasa de ticks cae a cero ($\nu \rightarrow 0$).

Control Predictivo No Lineal (NMPC)

El optimizador minimiza la matriz de costo J en tiempo real utilizando una matriz de nodos constante N , estirando el tiempo de predicción con $\Delta\tau_{pred} = \bar{T}/N$.

Acción de Control Continua: Para evitar la indeterminación del valor absoluto ($|U_t|$ en $x = 0$), la orden se divide en $U_t = u_{compra} - u_{venta}$ ($u \geq 0$). Los límites de capital operan como Box Constraints.

Sincronización Física (Partial Fills): Las órdenes se envían exclusivamente como IOC (Immediate or Cancel). El estado X_{t+1} se actualiza utilizando el volumen real ejecutado (U_{app}) validado por el vector Z_t , no el calculado, impidiendo fallas de inventario.

Fallback Determinista: Si el solver excede t_{max} , expulsa $U_t = 0$ (mantener posición).

Stack Tecnológico (Métodos Computacionales)

NMPC Solver: acados (vía Python a código C) utilizando iteraciones SQP (Programación Cuadrática Secuencial) para resolver la optimización en microsegundos. Alternativa: CasADi + IPOPT.

Autodiferenciación y EKF: JAX (compilación XLA). Computa Jacobianos exactos (jax.jacfwd) y mantiene las matrices de covarianza en la VRAM de la GPU.

EDP Minimax-Loeper: Numba (@cuda.jit). Discretización por mallas en diferencias finitas ejecutada nativamente en la RTX 3060 Ti para evadir cuellos de botella CPU-GPU.

4.6. Arquitectura Computacional: Integración de Tasa de Ticks

El sistema emplea un diseño de concurrencia para evitar que la varianza extrema en la llegada de órdenes (micro-estallidos) desestabilice los Jacobianos del NMPC.

1. Hilo Lento (Ingestión y Filtrado Espectral): Este proceso mantiene la conexión con el *websocket* del exchange y computa el estado físico del entorno sin latencia crítica.

- **Filtro de Tasa (ν):** Mantiene un *buffer* circular con los *timestamps* de los últimos N ticks. Aplica un filtro EMA estricto para calcular la tasa de llegada suavizada y la escala al dominio de años ($\frac{\text{Ticks}}{\text{Años}}$).
- **Métricas de Microestructura:** Computa la frecuencia del Micelio (ω_m) mediante el Periodograma de Lomb-Scargle y evalúa la falta de liquidez ($\lambda = \frac{\omega_m}{\Psi}$).
- **Memoria Compartida:** Escribe el vector de estado del entorno $[\nu, \lambda, \omega_m]$ en un bloque de memoria protegido por un *mutex* (exclusión mutua).

2. Hilo Rápido (Motor EKF y NMPC): Este proceso ejecuta el ciclo de control predictivo a una frecuencia estricta.

- **Lectura de Entorno:** Captura el vector $[\nu, \lambda, \omega_m]$ más reciente.
- **Actualización de Matrices (Autodiferenciación):** Calcula el presupuesto dinámico adversarial $c_{vol}^2 = k \cdot \omega_m \cdot \nu$. Inyecta el nuevo λ en el denominador de Loeper dentro de las matrices del modelo CasADi/acados.
- **Integración Temporal Dual:** Mide el tiempo físico Δt transcurrido desde el ciclo anterior. El *solver* avanza la matriz de difusión estocástica proyectando $\Delta \bar{T} = \nu \cdot \Delta t$ (ticks esperados), mientras integra los costos de capital (r_{USD}, q_{BTC}) directamente con Δt .
- **Ejecución:** Si el *solver* detecta que el denominador $1 - \lambda S^2 \Gamma \leq \epsilon$, aborta la búsqueda de gradiente por riesgo de singularidad y emite un comando de inacción ($U_t = 0$).

5. Introducción al Algoritmo MPC

El Control Predictivo Basado en Modelo (MPC) es una estrategia de control avanzado que utiliza un modelo matemático del sistema para simular su comportamiento futuro. Su núcleo es el **Control de Horizonte Recedente**, que consiste en los siguientes pasos continuos:

1. **Estimar:** Medir el estado actual del sistema (punto de partida).
2. **Predecir y Optimizar:** Simular múltiples futuros posibles (Horizonte de Predicción) y calcular la secuencia óptima de acciones de control.
3. **Ejecutar:** Aplicar únicamente la primera acción calculada y descartar el resto del plan.
4. **Avanzar y Repetir:** El tiempo avanza, se lee la nueva realidad del sistema y se repite la optimización desde cero.

5.1. La Función de Costo (J) y la Toma de Decisiones

Analogía: El Jugador de Ajedrez.

Así como en el ajedrez se evalúa el sacrificio de piezas para alcanzar el jaque mate, el MPC utiliza una Función de Costo (J) para encontrar el equilibrio entre llegar a la meta y el esfuerzo energético invertido.

La ecuación estándar es de naturaleza cuadrática para penalizar exponencialmente los errores grandes:

$$J = \sum_{i=0}^{N-1} (x_{k+i}^T Q x_{k+i} + u_{k+i}^T R u_{k+i}) \quad (6)$$

Donde:

- **Q (Peso del Error):** Penaliza la desviación del estado respecto a la meta (ej. temperatura de un horno industrial).
- **R (Peso del Control):** Penaliza el uso excesivo de energía o cambios bruscos en el actuador (ej. resistencia eléctrica).

El *solver* matemático iterará hasta encontrar la secuencia de acciones (u) que resulte en el valor de J más cercano a cero (el menor costo).

5.2. Manejo de Restricciones (Constraints)

Analogía: El Coche Autónomo Esquivando un Obstáculo.

El MPC permite programar inecuaciones que el sistema no debe violar, dividiéndolas en dos jerarquías:

- **Restricciones Duras (Hard Constraints):** Límites físicos inquebrantables, como el tope mecánico del ángulo del volante ($\delta \leq \delta_{max}$).
- **Restricciones Blandas (Soft Constraints):** Reglas operativas que se pueden romper en una emergencia, como salirse del carril ($Y \leq Y_{max} + \epsilon$).

Se introduce una **variable de holgura** (ϵ) en las restricciones blandas. Para evitar que el sistema la use sin justificación, se añade a la Función de Costo con un peso de castigo (P) extremadamente alto: $J = \dots + P \cdot \epsilon^2$. Así, el *solver* solo permitirá que $\epsilon > 0$ si es la única salida matemática para evitar violar una restricción dura (como chocar).

5.3. Perturbaciones Medibles (Eventos Futuros)

Analogía: La Casa Inteligente con Paneles Solares.

A diferencia de los controladores clásicos que son ciegos al futuro, el MPC puede incorporar pronósticos dentro de su simulación. El modelo matemático se expande:

$$x_{k+1} = Ax_k + Bu_k + Dd_k \quad (7)$$

Donde d_k es la perturbación (ej. pronóstico del clima). Si el MPC sabe que habrá una tormenta en los próximos días, modificará sus acciones de hoy (ej. cargar la batería al máximo aprovechando el sol actual). Gracias al horizonte recedente, si el pronóstico falla, el MPC corregirá el plan automáticamente en el siguiente instante temporal al actualizar la lectura de sus sensores.

5.4. Sistemas con Inercia y Tiempo Muerto (Delay)

Analogía: El Reactor Químico.

En sistemas donde la acción de control tarda en reflejarse en los sensores (tiempo muerto), el MPC es ideal porque planea a futuro evitando reacciones impulsivas y catastróficas. Para no saturar el procesador con cálculos innecesarios, se separan dos conceptos:

- **Horizonte de Predicción (H_p):** Cuánto tiempo simula el modelo hacia el futuro (debe ser mayor al tiempo muerto).
- **Horizonte de Control (H_c):** Cuántos cambios de acción calcula el *solver* dentro de ese futuro.

Al configurar, por ejemplo, $H_c = 1$ en un horizonte largo, el algoritmo asume que la válvula mantendrá esa misma posición durante el resto de la simulación, evaluando el impacto a largo plazo de esa única acción inmediata sin un alto costo computacional.

6. Interconexión del Algoritmo del Bot de Trading

Recopilando todos los conceptos anteriores, se procede a interconectar los nodos faltantes para que el algoritmo funcione bien, tanto computacional, como lógica, como por análisis dimensional:

Sintonización de la Matriz Q

La volatilidad aislada en la primera IMF, $c_1(\overline{T})$, se utiliza para calcular la varianza local e inyectar ese escalar dinámicamente en la matriz Q del Filtro de Kalman.

6.1. Función de Costo y Penalización Dinámica del Micelio

El corazón del algoritmo de Control Predictivo No Lineal (NMPC) radica en la minimización de una función de costo J proyectada sobre un horizonte de predicción de N pasos. La optimización busca un equilibrio entre maximizar la rentabilidad del portafolio, manteniéndose cerca del Delta neutral óptimo, y minimizar los costos de ejecución sujetos al impacto de mercado.

Para dotar al sistema de una aversión al riesgo endógena, se emplea la métrica de estabilidad de la Canción del Micelio (Ω) como un modulador dinámico de la matriz de penalización, en lugar de una variable de estado predictiva.

La función de costo escalar a minimizar en cada iteración temporal se define como:

$$\min_U J = \sum_{k=0}^{N-1} (e_k^T Q e_k + U_k^T R(\Omega) U_k) + e_N^T P e_N$$

Donde:

- e_k : Es el vector de error del estado en el paso k (desviación respecto al Delta óptimo de cobertura).

- Q y P : Son las matrices definidas positivas que penalizan el error de estado en la trayectoria y en el nodo terminal, respectivamente.
- U_k : Es la acción de control neta calculada como $U_k = u_{compra} - u_{venta}$.

La Matriz de Penalización Modulada $R(\Omega)$

Para evitar la indeterminación de derivadas y saltos no diferenciables en *solvers* basados en SQP, se descarta el uso directo de normas L_1 . En su lugar, el costo de ejecución emplea una norma cuadrática estricta cuya resistencia (peso R) varía en función de la estabilidad de la estructura del mercado detectada por el Hilo Lento.

Definimos el peso de penalización efectivo como:

$$R(\Omega) = R_{base} + \kappa \cdot \Omega^2$$

Donde:

- R_{base} : Costo estático por comisiones del *exchange* (e.g., *Taker fee*).
- κ : Factor de escalado (hiperparámetro de aversión a la inestabilidad). Para mantener la estricta consistencia dimensional con el costo base, posee unidades de $\left[\frac{\text{Ticks}^4}{\text{BTC}^2}\right]$.
- Ω^2 : Cuadrado de la tasa de cambio de la estabilidad del mercado $(\frac{\partial \Psi}{\partial T})^2$. Al elevarlo al cuadrado, se garantiza la convexidad, penalizando por igual las inyecciones y extracciones violentas de liquidez estructural.

Comportamiento del Solver

Bajo esta arquitectura de penalización, el optimizador evalúa el entorno antes de inyectar el control:

1. **Mercado Armónico** ($\Omega \approx 0$): La relación flujo/precio es constante. $R(\Omega) \approx R_{base}$ y el NMPC resuelve la matriz dinámica ejecutando órdenes libremente para equilibrar el portafolio y maximizar el modelo Minimax.
2. **Transición de Fase** ($\Omega \gg 0$): Ocurre una anomalía (ej. barrido de liquidez institucional). La penalización $R(\Omega)$ diverge hacia el infinito. El costo en la función J por ejecutar U_k se vuelve tan asimétricamente destructivo que la derivada del Jacobiano empuja inmediatamente el vector de decisión hacia el origen, forzando la inacción ($U_k = 0$) hasta que la superficie matemática se estabilice.

6.2. Restricciones de Caja Dinámicas (Box Constraints) Asimétricas

Para evitar el “problema del inventario tóxico” (congelamiento del sistema con exposición direccional abierta durante un colapso estructural), la acción de con-

trol se descompone estrictamente en componentes no negativos:

$$U_t = u_{compra} - u_{venta} \quad \text{sujeto a} \quad u_{compra} \geq 0, u_{venta} \geq 0$$

En lugar de imponer un límite de capital estático, el sistema emplea Restricciones de Caja Dinámicas en el optimizador NMPC. Estos límites superiores ($\bar{u}_{compra}, \bar{u}_{venta}$) se reescriben algorítmicamente en la interfaz de Python/C (modificando el vector `ubu` de `acados`) en cada iteración del Hilo Rápido, previo al cálculo del paso SQP.

Las restricciones dependen de la magnitud del inventario actual I_t (medido en BTC) y del estado de estabilidad del mercado evaluado por el Micelio (Ω) frente a un umbral crítico Ω_{crit} .

La formulación algorítmica de los límites superiores se define como:

1. **Estado Armónico** ($|\Omega| < \Omega_{crit}$): Se permite la bidireccionalidad completa hasta el límite de capital disponible en la billetera (C_{max}) y el riesgo máximo permitido por operación (U_{max}).

$$\bar{u}_{compra} = \min(C_{max}, U_{max})$$

$$\bar{u}_{venta} = \min(C_{max}, U_{max})$$

2. **Transición de Fase o Caos** ($|\Omega| \geq \Omega_{crit}$): Se activa la relajación asimétrica para forzar el vaciado direccional del inventario I_t , bloqueando matemáticamente promediar a la baja. Se emplea la función signo analítica $\text{sgn}(x)$ para anular los vectores de riesgo:

- Si el sistema está en posición Larga ($I_t > 0$):

$$\bar{u}_{compra} = 0$$

$$\bar{u}_{venta} = U_{max}$$

Efecto matemático: El *solver* solo puede explorar soluciones donde u_{venta} reduzca el error de inventario hacia cero.

- Si el sistema está en posición Corta ($I_t < 0$):

$$\bar{u}_{compra} = U_{max}$$

$$\bar{u}_{venta} = 0$$

Efecto matemático: Se bloquea el apalancamiento negativo adicional, permitiendo únicamente recompras de cobertura.

Al externalizar esta lógica condicional fuera del grafo de diferenciación de `CasADi`, se preserva la convexidad estricta de las matrices Hessianas en la formulación de optimización, previniendo el colapso numérico del *solver* y permitiendo una adaptación instantánea al riesgo estructural exógeno.

Aversión Preventiva de Inventario en la Función de Costo

El mecanismo de restricciones de caja asimétricas actúa como un seguro de última instancia, pero la teoría de control óptimo exige mitigar el riesgo direccional *antes* de que ocurra el colapso estructural. Para lograr esto, se incorpora un término de penalización cuadrática por inventario abierto (I_k) directamente en la función de costo J del optimizador NMPC.

Esta modificación obliga al sistema a desprenderse de las posiciones gradualmente a medida que la inestabilidad de la Canción del Micelio (Ω) aumenta, asegurando que cuando el bloqueo de órdenes sea inminente, la exposición neta (Delta) ya sea cercana a cero.

La función de costo expandida se redefine como:

$$\min_U J = \sum_{k=0}^{N-1} (e_k^T Q e_k + U_k^T R(\Omega) U_k + q_{inv}(\Omega) \cdot I_k^2) + e_N^T P e_N$$

Donde el factor de aversión al inventario $q_{inv}(\Omega)$ se modela de forma dinámica:

$$q_{inv}(\Omega) = q_{base} + \mu \cdot \Omega^2$$

- q_{base} : Costo base asintótico por mantenimiento de inventario (puede asociarse al costo de oportunidad o *funding rate* del activo).
- μ : Hiperparámetro de sensibilidad al riesgo de exposición.
- I_k^2 : Nivel de inventario al cuadrado. Garantiza la convexidad matemática de la penalización, castigando asimétricamente la retención de grandes volúmenes frente a posiciones fraccionarias.

Dinámica de Vaciado Preventivo: A medida que la tasa de cambio de la estabilidad estructural se altera ($\Omega \gg 0$), el parámetro q_{inv} crece agresivamente. Al proyectar la trayectoria futura en su horizonte de predicción, el *solver* SQP evaluará que la penalización por retener el inventario I_k^2 es superior al costo por impacto de mercado y comisiones ($R(\Omega)$).

En consecuencia, el vector de control óptimo inyectará automáticamente órdenes para aplanar la cartera, ejecutando un *vaciado preventivo* mucho antes de que la singularidad de Loeper alcance su límite crítico (Ω_{crit}) y detone las restricciones de inacción total.

6.3. Límites de Integración de Volumen y Estados del Mercado

Para garantizar la coherencia física del modelo $\Psi = \frac{\Phi}{\Delta S}$, el cálculo del desplazamiento de precio y la acumulación de volumen deben evaluarse sobre el mismo dominio temporal.

Dado que la referencia de precio S_{ref} se actualiza dinámicamente en cada nodo estructural $\bar{T}_{nodo}$, la acumulación de volumen neto no puede ser un sumatorio histórico continuo hacia el infinito. Si se mantuviera así, Ψ divergiría matemáticamente en cada nuevo ciclo.

Por lo tanto, se define el volumen neto del ciclo actual como una integral discreta por tramos, la cual se reinicia a cero tras cruzar cada nodo de fase de la Transformada de Hilbert:

$$\sum Q(\bar{T}) = \sum_{k=\bar{T}_{nodo}}^{\bar{T}} Q_k$$

Redefinición de los Estados de Mercado (Intensidad vs. Dirección)

En la dinámica de fluidos financieros, un mercado en distribución masiva (ventas) genera un volumen neto $\sum Q$ altamente negativo, lo que resulta en un flujo Φ negativo. Para medir correctamente el “esfuerzo” del mercado sin que los signos anulen la evaluación lógica, se debe utilizar la intensidad absoluta del flujo $|\Phi|$ evaluada contra un umbral de actividad promedio $\bar{\Phi}$.

Los estados estructurales del activo se redefinen de la siguiente manera:

1. $(|\Phi| > \bar{\Phi}) \wedge (\Delta S < 0)$: Alta intensidad de flujo, pero el activo cotiza por debajo de su nodo de referencia (Absorción de demanda o Distribución Institucional).
2. $(|\Phi| > \bar{\Phi}) \wedge (\Delta S > 0)$: Alta intensidad de flujo y desplazamiento positivo (Acumulación o Tendencia alcista confirmada).
3. $\frac{\partial \Psi}{\partial \bar{T}} = 0$: El mercado mantiene su estabilidad estructural (esfuerzo proporcional al resultado).
4. $(|\Phi| > \bar{\Phi}) \wedge (\Delta S \approx 0)$: Alto flujo de volumen sin desplazamiento direccional del precio (Modelo Coca-Cola: máxima absorción de liquidez, alerta de transición de fase).
5. $(|\Phi| < \bar{\Phi}) \wedge (\Delta S > 0)$: Baja intensidad de flujo, pero el activo logra desplazarse positivamente (Modelo Ferrari: alta exclusividad y escasa resistencia en el libro de órdenes).

6.4. Desacoplamiento Computacional y Horizonte Reciente

Obligar al optimizador SQP del Control Predictivo No Lineal (NMPC) a resolver la Ecuación Diferencial Parcial (EDP) estocástica de Minimax-Loeper en cada uno de sus nodos de predicción futura, y en cada iteración de descenso de gradiente, resulta en un problema computacional de complejidad $O(N^3)$. Esto viola la restricción de latencia estricta del Hilo Rápido ($t_{max} = T_{min}/2$).

Para evitar el colapso numérico sin caer en la miopía algorítmica (optimizar solo el presente ignorando las consecuencias futuras), se implementa una separación de responsabilidades entre el procesador (CPU) y el acelerador gráfico (GPU), orquestada bajo el principio del **Horizonte Recedente**.

1. **Superficie de Interpolación Estática (GPU):** Al inicio de cada ciclo de control en t_0 , el núcleo compilado con Numba `@cuda.jit` resuelve la EDP de Loeper utilizando el estado actual del mercado. El resultado no es un escalar, sino una matriz tridimensional de alta resolución que mapea la convexidad $\Gamma(S, t)$ para todos los precios posibles y todos los instantes futuros dentro de la ventana de predicción $\Delta\tau_{pred}$.
2. **Optimización de Trayectoria (CPU):** El *solver* (CasADi/acados) inicia su búsqueda de gradiente minimizando la función de costo J a lo largo de N pasos hacia el futuro. En lugar de recalcular la física del mercado, el optimizador simplemente consulta (vía interpolación B-spline rápida) las coordenadas (S_k, t_k) en la malla precalculada por la GPU. Esto reduce la carga computacional a $O(1)$ por nodo, asegurando convergencia en microsegundos.
3. **Ejecución por Horizonte Recedente:** A pesar de que el *solver* genera un vector óptimo con un plan estratégico para los próximos N pasos $U = [U_0, U_1, \dots, U_{N-1}]$, el sistema es estocástico. El bot inyecta estrictamente la primera acción U_0 en el *exchange* y descarta el resto. En el siguiente *tick*, los datos se actualizan, la GPU genera una nueva malla y la CPU planifica una nueva trayectoria iterativa.

Esta arquitectura garantiza que el bot posea profundidad estratégica a largo plazo para esquivar la “Singularidad de Loeper”, manteniendo simultáneamente una adaptabilidad táctica instantánea ante el ruido de microestructura detectado por la Canción del Micelio.

6.5. Asincronía y Compensación de Latencia Estocástica

La arquitectura del sistema impone una restricción de tiempo real estricta para el Hilo Rápido. Sin embargo, el flujo de datos del *exchange* a través del *websocket* está sujeto a anomalías de red, específicamente la pérdida temporal de paquetes (*dropouts*) y la variación del retardo (*jitter*). Si estas discrepancias temporales se introducen directamente en la fase de corrección del Filtro de Kalman (EAKF), generarán derivadas de velocidad artificialmente infinitas, corrompiendo la matriz de estado y desestabilizando el controlador NMPC.

Para preservar la integridad matemática del estimador de estados, se implementa un mecanismo de compensación basado en el sello de tiempo (*timestamp*) de origen de cada medición z_k .

1. Ausencia de Datos (*Dropouts*)

Si el límite de tiempo de iteración del NMPC expira antes de recibir un nuevo paquete válido, se anula la fase de actualización del EAKF. Matemáticamente, la Ganancia de Kalman se fuerza a una matriz nula ($K_k = 0$). El vector de estado y la covarianza del error se actualizan basándose exclusivamente en el modelo cinemático de transición:

$$x_k = x_{pred}$$

$$P_k = P_{pred}$$

Bajo esta condición, el sistema confía en la inercia de la predicción física y la incertidumbre $Tr(P)$ crece simétricamente debido a la matriz Q , reflejando la pérdida temporal de visión del mercado.

2. Penalización Exponencial por Retardo (*Jitter*)

Cuando se recibe una medición válida, se calcula su retardo de red τ_d como la diferencia entre el tiempo de reloj local del servidor y el *timestamp* del *exchange*: $\tau_d = t_{local} - t_{orig}$.

Si τ_d supera un umbral de tolerancia máximo τ_{max} , el paquete se descarta por pérdida de relevancia táctica. Si $\tau_d \leq \tau_{max}$, el dato se asimila, pero su autoridad se degrada dinámicamente.

Se modula el coeficiente de ruido de observación del precio (r_S) dentro de la matriz R utilizando una función de decaimiento exponencial:

$$R_{eff}^{(1,1)} = r_{S,base} \cdot e^{\beta\tau_d}$$

Donde $\beta > 0$ es el factor de sensibilidad a la latencia. Esta formulación asegura que a medida que el retardo de un paquete aumenta, su varianza de observación tiende al infinito. Consecuentemente, el filtro de Kalman absorbe el impacto de un salto de precio retrasado reduciendo drásticamente su ganancia K , previniendo la generación de gradientes de velocidad ilusorios.

7. Flujo Lógico del Algoritmo

Se procede a detallar los pasos lógicos que el algoritmo iterará:

7.1. Inicialización y Purgado (Burn-in) del Sistema Estocástico

El arranque en frío del Filtro de Kalman Extendido Adaptativo (EAKF) presenta un periodo de inestabilidad transitoria. Durante los primeros minutos de ejecución, las estimaciones de los estados ocultos (la velocidad v y el residuo

macro R_n) divergen de la realidad física, dado que el sistema inicializa con suposiciones. Si el optimizador NMPC recibiera estos datos transitorios, el error de estimación se amplificaría, detonando falsos positivos en la estabilidad Ω y ejecutando órdenes catastróficas.

Para mitigar esto, se implementa una fase obligatoria de Purgado (*Burn-in Period*), donde el NMPC es forzado a un estado de inacción absoluta ($U_t = 0$), operando el sistema como un mero observador (*shadow mode*) hasta que el filtro converja.

Configuración de Matrices Iniciales

Dado el vector de estado aumentado $x = [S, v, R_n]^T$, las matrices se inicializan bajo un esquema de asimetría de confianza:

- **Ruido de Observación (R):** $R = \text{diag}([r_S, r_{EMD}])$. Se asigna un valor minúsculo a r_S basado en la resolución del *tick* del *exchange*, y un valor empírico a r_{EMD} derivado del error de ajuste del algoritmo de Hilbert-Huang.
- **Ruido de Proceso Base (Q_0):** Se inicializa con valores mínimos de varianza térmica antes de la modulación dinámica de la Canción del Micelio.
- **Covarianza del Error Inicial (P_0):** $P_0 = \text{diag}([p_S, p_v, p_{R_n}])$. El valor p_S se inicializa cercano a cero (alta certeza en la primera lectura de precio). En contraste, p_v y p_{R_n} se inicializan con escalares masivos ($p_v, p_{R_n} \rightarrow \infty$). Esta alta incertidumbre matemática fuerza a la Ganancia de Kalman a maximizarse en los primeros instantes, obligando al sistema a corregir los estados ocultos agresivamente usando los datos crudos entrantes.

Condición de Convergencia: La Traza de P

El fin del periodo de purgado no se determina por un temporizador cronológico arbitrario, sino por la estabilidad de la matriz de covarianza.

Se define la incertidumbre total del sistema en el instante k como la traza de la matriz P :

$$\text{Incertidumbre}_k = \text{Tr}(P_k) = \sum_i P_k^{(i,i)}$$

Al iniciar el algoritmo, $\text{Tr}(P_0)$ es gigante. Con cada iteración, la asimilación de datos contrae el cono de incertidumbre. El sistema se considera convergente y purgado cuando la derivada de la traza entra en una banda muerta estática (estado estacionario):

$$\left| \frac{\text{Tr}(P_k) - \text{Tr}(P_{k-1})}{\Delta t} \right| \leq \epsilon_{burn}$$

Una vez cumplida esta condición durante una ventana continua de N *ticks*, el Hilo Rápido levanta el bloqueo de las restricciones de caja del optimizador

CasADi/acados, permitiendo el despliegue del portafolio bajo la supervisión de la Ecuación Minimax-Loeper.

7.2. Flujo Lógico de Orquestación y Ciclo de Ejecución

Para evitar el bloqueo de recursos computacionales y garantizar latencias en el orden de los microsegundos, el sistema distribuye su carga matemática en una arquitectura asíncrona de memoria compartida. Asumiendo que la **Fase de Inicialización y Purgado (Burn-in)** ha convergido exitosamente y la traza de la covarianza $Tr(P)$ es estable, el ciclo operativo del algoritmo (Horizonte Recedente) ejecuta los siguientes pasos lógicos iterativos:

1. **Ingesta Asíncrona y Saneamiento (Hilo Rápido):** El sistema recibe un nuevo paquete de datos del *exchange* con el precio crudo P_{spot} y el volumen transado Q_k . Se evalúa el retardo de red estocástico τ_d . Si $\tau_d > \tau_{max}$, el paquete se descarta. Si es válido, se calcula la penalización exponencial y se actualiza la matriz de observación R_{eff} .
2. **Análisis Estructural y Micelio (Hilo Lento - Background):** Operando a una frecuencia menor sobre la ventana temporal adaptativa W_k , el núcleo de análisis procesa la serie de tiempo:
 - Ejecuta la Descomposición Empírica de Modos (EMD) y la Transformada de Hilbert para extraer las frecuencias estructurales f_i y el residuo macroeconómico R_n .
 - Identifica los nodos de fase $\bar{T}_{nodo}$ y actualiza la referencia de precio S_{ref} , reiniciando la integral de volumen $\sum Q$.
 - Calcula las variables de estado de la Canción del Micelio: el desplazamiento direccional ΔS , el flujo absoluto $|\Phi|$ y la derivada de estabilidad estructural Ω .
 - Actualiza estos tensores en la memoria compartida, inyectando la frecuencia ponderada ω_m en la matriz de varianza del proceso Q .
3. **Estimación Cinemática del Estado (EAKF):** El Hilo Rápido lee los últimos parámetros actualizados por el Hilo Lento y ejecuta la corrección del Filtro de Kalman Extendido:
 - *Predicción:* $x_{pred} = Ax_{ant}$
 - *Corrección:* Se asimila P_{spot} sujeto al peso de R_{eff} . El filtro emite el vector de estado purificado que contiene el precio verdadero S y su velocidad instantánea v .
4. **Superficie de Loeper y Resolución de EDP (GPU):** El núcleo paralelo compilado (@cuda.jit) recibe el precio y la velocidad filtrados. Resuelve la Ecuación Diferencial Parcial estocástica de Minimax-Loeper

proyectando una malla estática tridimensional. Esta malla contiene la convexidad $\Gamma(S, t)$ y la liquidez adversaria λ para todo el horizonte temporal futuro $\Delta\tau_{pred}$.

5. **Configuración del Controlador de Riesgo (CPU):** La métrica de estabilidad Ω modula los hiperparámetros de control preventivo antes de la optimización:
 - Actualiza la penalización de acción $R(\Omega)$ y la aversión al inventario $q_{inv}(\Omega)$ en la función de costo J .
 - Modifica algorítmicamente las restricciones de caja asimétricas superiores e inferiores (ubu, lbu), permitiendo cierres de inventario direccional I_t pero bloqueando apalancamientos destructivos en caso de que $|\Omega| \geq \Omega_{crit}$.
6. **Optimización de Trayectoria Predictiva (NMPC):** El *solver* SQP (acados/CasADi) busca la secuencia óptima de decisiones de compra/venta minimizando J a lo largo de N pasos futuros. Durante el descenso de gradiente, el *solver* no calcula derivadas físicas complejas; simplemente interpola las coordenadas (S_k, t_k) en la malla de la GPU, reduciendo la complejidad del cálculo predictivo a $O(1)$ por nodo. Se genera el vector óptimo de acciones: $U = [U_0, U_1, \dots, U_{N-1}]$.
7. **Ejecución Táctica y Desplazamiento:** En cumplimiento con la teoría del Horizonte Recedente, el sistema aísla e inyecta únicamente la primera acción U_0 en el motor de ejecución de órdenes del *broker*. Los pasos restantes $U_{1\dots N-1}$ son descartados. El reloj avanza, el estado actual pasa a ser x_{ant} , y el sistema retorna al Paso 1.

7.3. Formulación Explícita del Filtro de Kalman Adaptativo (EAKF)

El sistema se define sobre un espacio de estados de dimensión $n = 3$ y un espacio de medición de dimensión $m = 2$. En cada paso temporal discreto k , con un intervalo Δt entre iteraciones del Hilo Rápido, las matrices se estructuran de la siguiente manera:

7.3.1. 1. Vectores de Estado y Medición

El vector de estado oculto $x_k \in \mathbb{R}^{3 \times 1}$ y el vector de medición $z_k \in \mathbb{R}^{2 \times 1}$ se definen como:

$$x_k = \begin{bmatrix} S_k \\ v_k \\ R_{n,k} \end{bmatrix}, \quad z_k = \begin{bmatrix} P_{spot,k} \\ R_{EMD,k} \end{bmatrix}$$

Donde S_k es el precio verdadero, v_k la velocidad instantánea, $R_{n,k}$ el residuo macroscópico, $P_{spot,k}$ el precio reportado por el *broker*, y $R_{EMD,k}$ la última actualización del residuo generada por el Hilo Lento.

7.3.2. 2. Matrices de Dinámica del Sistema

Matriz de Transición ($A \in \mathbb{R}^{3 \times 3}$): Define la cinemática de las variables asumiendo velocidad constante en el micro-intervalo Δt .

$$A = \begin{bmatrix} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matriz de Observación ($H \in \mathbb{R}^{2 \times 3}$): Mapea el espacio de estado al espacio de medición (hace invisibles las variables que no se pueden medir, como v_k).

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

7.3.3. 3. Matrices de Ruido y Adaptación Dinámica

Matriz de Ruido de Observación ($R_k \in \mathbb{R}^{2 \times 2}$): Se modela como una matriz diagonal. Incorpora la penalización por latencia de red estocástica (Jitter).

$$R_k = \begin{bmatrix} r_{S,base} \cdot e^{\beta \tau_{d,k}} & 0 \\ 0 & r_{EMD} \end{bmatrix}$$

Parámetros: $r_{S,base}$ es la varianza mínima del *tick size*, $\tau_{d,k}$ es el retardo del paquete en milisegundos, β es el factor de sensibilidad al retardo, y r_{EMD} es la varianza histórica de la Transformada de Hilbert.

Matriz de Ruido del Proceso ($Q_k \in \mathbb{R}^{3 \times 3}$): Modulada exógenamente por el espectro de Hilbert y la inestabilidad de volumen del Micelio para expandir la ganancia ante anomalías estructurales.

$$Q_k = \rho_k \cdot \begin{bmatrix} q_S & 0 & 0 \\ 0 & q_v & 0 \\ 0 & 0 & q_{R_n} \end{bmatrix}, \quad \text{donde} \quad \rho_k = 1 + \gamma_\omega |\omega_m| + \gamma_Q \left| \sum Q \right|$$

Parámetros: q_S, q_v, q_{R_n} son las varianzas base del modelo térmico. ρ_k es el escalar de expansión dinámica alimentado por el Hilo Lento (frecuencia ponderada ω_m y volumen neto $\sum Q$), y γ_ω, γ_Q son sus respectivos pesos de ponderación.

7.3.4. 4. Matriz de Covarianza Inicial (Burn-in)

Matriz $P_0 \in \mathbb{R}^{3 \times 3}$: Diseñada para forzar la convergencia rápida de los estados ocultos.

$$P_0 = \begin{bmatrix} p_S & 0 & 0 \\ 0 & p_v & 0 \\ 0 & 0 & p_{R_n} \end{bmatrix}$$

Condición inicial: $p_S \approx 0$ (alta certeza en el primer precio recibido), mientras que $p_v, p_{R_n} \approx 10^6$ (incertidumbre masiva inicial forzando asimilación de datos).

7.3.5. 5. Ciclo de Ecuaciones Recursivas (Implementación)

En cada iteración k , el algoritmo ejecuta secuencialmente:

Fase de Predicción:

$$\begin{aligned} x_{k|k-1} &= Ax_{k-1|k-1} \\ P_{k|k-1} &= AP_{k-1|k-1}A^T + Q_k \end{aligned}$$

Fase de Corrección (Actualización): Se evalúa la existencia del paquete (*Dropout bypass*):

- Si NO hay dato ($z_k = \emptyset$):

$$K_k = \mathbf{0}_{3 \times 2}, \quad x_{k|k} = x_{k|k-1}, \quad P_{k|k} = P_{k|k-1}$$

- Si hay dato válido ($z_k \neq \emptyset$):

$$\begin{aligned} \tilde{y}_k &= z_k - Hx_{k|k-1} && \text{(Innovación)} \\ S_k &= HP_{k|k-1}H^T + R_k && \text{(Covarianza de la Innovación)} \\ K_k &= P_{k|k-1}H^T S_k^{-1} && \text{(Ganancia de Kalman)} \\ x_{k|k} &= x_{k|k-1} + K_k \tilde{y}_k && \text{(Actualización de Estado)} \\ P_{k|k} &= (I_{3 \times 3} - K_k H)P_{k|k-1} && \text{(Actualización de Covarianza)} \end{aligned}$$

Nota computacional: Se emplea $(I - K_k H)P_{k|k-1}$ en su forma de Joseph o simetrizada en código $(\dots + K_k R_k K_k^T)$ si ocurren problemas de estabilidad numérica (pérdida de simetría definida positiva en P).

7.4. Formulación Explícita de la EDP de Minimax-Loeper (GPU)

Para que el optimizador NMPC evalúe el impacto de mercado sin recalculer la física del sistema en cada iteración, se emplea el procesador gráfico (GPU) para resolver la EDP no lineal de Loeper. El resultado es un tensor estático que mapea el riesgo estructural del mercado en el espacio de precios y el horizonte de predicción.

7.4.1. 1. Dominio Espacio-Temporal y Bifurcación Temporal (Mesh Grid)

Se define una malla bidimensional sobre la cual el kernel de CUDA calculará la solución numérica mediante diferencias finitas, aislando el componente continuo del tiempo dual:

- **Eje de Precio (S):** Rango operativo centrado en S_k con un margen de $\pm \Delta S_{max}$. El vector discreto de dimensión N_S es: $\mathcal{S} = \{S_0, S_1, \dots, S_{N_S-1}\}$.

- **Eje de Tiempo Inverso (τ):** Para evitar la divergencia anti-difusiva de la EDP (cuyo planteamiento analítico exige retroceder desde la madurez del contrato), el eje se define como el tiempo remanente hasta el final del horizonte predictivo ($\tau = t_{pred} - t$), expresado estrictamente en años (Años). El vector de dimensión N_t marcha desde el futuro ($j = N_t - 1$) hacia el presente ($j = 0$).

Los pasos espaciales y temporales de la malla son $ds = \frac{2\Delta S_{max}}{N_S - 1}$ y $dt = \frac{\Delta\tau_{pred}(\text{años})}{N_t - 1}$. Al invertir la marcha iterativa, el problema parabólico se vuelve bien planteado, validando el uso del criterio de Courant-Friedrichs-Lewy (CFL) para garantizar la estabilidad del esquema explícito en la GPU.

7.4.2. 2. Variables de Estado del Modelo de Loeper

El objetivo numérico de la GPU es calcular la función de valor $U(S, t)$ y extraer su segunda derivada espacial (la convexidad de la cartera frente al impacto de mercado).

- $U_{i,j}$: Valor aproximado de U en la coordenada (S_i, t_j) .
- λ : Coeficiente de iliquidez del libro de órdenes (profundidad de mercado adversaria).
- $\Gamma_{i,j}$: Convexidad local, calculada discretamente mediante diferencias centrales finitas:

$$\Gamma_{i,j} = \frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{(ds)^2}$$

7.4.3. 3. Condición Terminal, Singularidad y Marcha Inversa

Dado que la formulación Minimax-Loeper pertenece a la familia de ecuaciones de *Pricing* retroactivas, la resolución exige anclar el *payoff* futuro en el nodo terminal ($j = N_t - 1$) e integrar iterativamente hacia el instante presente ($j = 0$), donde el optimizador NMPC interpolará los tensores.

Condición Terminal (Futuro, $\tau = 0$):

$$U_{i,N_t-1} = \frac{1}{2}\gamma_0(S_i - S_{ref})^2$$

Marcha Inversa (*Backward Stepping*): Para cada paso temporal decreciente (desde $j = N_t - 1$ hasta $j = 1$), cada hilo de la GPU evalúa el factor de fricción crítica y actualiza el nodo temporal inmediatamente anterior ($j - 1$):

$$\mathcal{D}_{i,j} = 1 - \lambda \cdot S_i^2 \cdot \Gamma_{i,j}$$

$$costos_{i,j} = q_{BTC}U_{i,j} - (r_{USD} - q_{BTC})S_i \frac{U_{i+1,j} - U_{i-1,j}}{2ds}$$

Integrando en retroceso:

$$U_{i,j-1} = U_{i,j} + dt \cdot \left(\frac{1}{2} \frac{c_{vol}^2 S_i^2 \Gamma_{i,j}}{\mathcal{D}_{i,j}} - \text{costos}_{i,j} \right)$$

Convergencia: Este esquema retrospectivo suprime la inestabilidad topológica inducida por el error de truncamiento del *mesh*. Consecuentemente, el colapso del denominador ($\mathcal{D}_{i,j} \leq \epsilon$) ocurrirá exclusivamente por estrés de liquidez macroestructural, entregando a la memoria compartida de la CPU una matriz de convexidad ($\mathbf{M}_\Gamma$) matemáticamente irrefutable.

7.4.4. 4. Salida Computacional: El Tensor de Interpolación

Si en cualquier coordenada la función de fricción cruza el límite crítico ($\mathcal{D}_{i,j} \rightarrow 0$), la volatilidad efectiva colapsa y el sistema anota una singularidad. La GPU empaqueta estos resultados y devuelve a la memoria compartida (RAM) una superficie bidimensional estática (Matriz $\mathbf{M}_\Gamma \in \mathbb{R}^{N_S \times N_t}$):

$$\mathbf{M}_\Gamma = \begin{bmatrix} \Gamma_{0,0} & \Gamma_{0,1} & \dots & \Gamma_{0,N_t-1} \\ \Gamma_{1,0} & \Gamma_{1,1} & \dots & \Gamma_{1,N_t-1} \\ \vdots & \vdots & \ddots & \vdots \\ \Gamma_{N_S-1,0} & \Gamma_{N_S-1,1} & \dots & \Gamma_{N_S-1,N_t-1} \end{bmatrix}$$

Es esta matriz $\mathbf{M}_\Gamma$ (junto con el tensor de iliquidez λ) la que el optimizador NMPC interpolará durante su descenso de gradiente mediante *B-splines* de CasADi, buscando la trayectoria U_t óptima sin tener que resolver ecuaciones diferenciales numéricamente.

7.5. Formulación Explícita del Controlador NMPC (acados/CasADi)

El optimizador resuelve la trayectoria de ejecución sobre un horizonte de predicción de N pasos. Para mantener el Jacobiano disperso (*sparse*) y garantizar convergencia convexa, el sistema desacopla la predicción del mercado de la dinámica del portafolio. Las variables del Filtro de Kalman y la malla de la GPU se inyectan como parámetros numéricos fijos (TVP) en cada nodo de la predicción.

7.5.1. 1. Vectores de Decisión y Estado del Controlador

El vector de decisión de control $u_k \in \mathbb{R}^{2 \times 1}$ se descompone en flujos unidireccionales para permitir restricciones asimétricas:

$$u_k = \begin{bmatrix} u_{compra,k} \\ u_{venta,k} \end{bmatrix} \quad \text{sueto a} \quad \mathbf{0} \leq u_k \leq \begin{bmatrix} \bar{u}_{compra} \\ \bar{u}_{venta} \end{bmatrix}$$

Dado que el Delta óptimo de cobertura (Δ_k) no es alterado de forma directa por las órdenes de mercado del bot, sino que es una propiedad dictada por la cinemática del precio y la EDP de Loeper, este se extrae de la matriz de transición. En su lugar, Δ_k se inyecta en el *solver* como un Parámetro Variable en el Tiempo (TVP) exógeno para cada nodo futuro de predicción.

Por lo tanto, el vector de estado interno del portafolio $x_k^c \in \mathbb{R}^{1 \times 1}$ rastrea de manera exclusiva la acumulación física del activo base (Inventario):

$$x_k^c = [I_k]$$

La dinámica del estado (Ecuación de Transición del NMPC) se reduce a una conservación estricta de masa, eliminando derivadas inexistentes y aligerando la carga computacional del integrador *Runge-Kutta*:

$$I_{k+1} = I_k + u_{compra,k} - u_{venta,k}$$

Para enlazar la dinámica del inventario físico con el objetivo financiero de mitigación de riesgo, y para mantener la coherencia dimensional con la matriz de costo $Q \in \mathbb{R}^{2 \times 2}$, se define el vector de evaluación penalizado $\chi_k \in \mathbb{R}^{2 \times 1}$:

$$\chi_k = \begin{bmatrix} e_k \\ I_k \end{bmatrix}$$

Donde $e_k = I_k - \Delta_k$ representa el error de cobertura contra el Delta objetivo exigido por el mercado.

7.5.2. 2. Matrices de Penalización (Q , R , P)

Para garantizar la convexidad estricta y evitar que el *solver* diverja, las matrices de ponderación se diseñan como matrices diagonales definidas positivas. Su formulación explícita integra la inestabilidad de la Canción del Micelio (Ω) y el impacto de mercado interpolado de la malla de Loeper (Γ_k).

Matriz de Costo de Estado ($Q \in \mathbb{R}^{2 \times 2}$): Penaliza el desvío del Delta y activa el vaciado preventivo de inventario:

$$Q(\Omega) = \begin{bmatrix} q_\Delta & 0 \\ 0 & q_{base} + \mu\Omega^2 \end{bmatrix}$$

Donde: q_Δ es la rigidez del seguimiento de la cobertura, y el término inferior es el factor $q_{inv}(\Omega)$ que castiga mantener inventario durante transiciones de fase.

Matriz de Costo de Control ($R \in \mathbb{R}^{2 \times 2}$): Penaliza el esfuerzo de ejecución. Se modula por el riesgo del entorno y por el impacto auto-inducido. Evaluando las variables exógenas en el nodo k :

$$R_k(\Omega, S_k, \Gamma_k) = \begin{bmatrix} R_{eff,k} & 0 \\ 0 & R_{eff,k} \end{bmatrix}$$

$$\text{Con: } R_{eff,k} = R_{base} + \kappa\Omega^2 + f_{Loeper}(S_k^2, \lambda, \Gamma_k)$$

Física de la matriz: Si Ω crece, la ejecución general se bloquea. Si la interpolación en la matriz de la GPU dicta que la convexidad Γ_k genera un impacto destructivo, la función f_{Loeper} dispara el valor de $R_{eff,k}$ en ese nodo temporal específico, forzando al optimizador a diferir la compra/venta a un nodo futuro donde el impacto sea menor.

Matriz de Costo Terminal ($P \in \mathbb{R}^{2 \times 2}$): Asegura la estabilidad de Lyapunov al final del horizonte temporal N :

$$P = \begin{bmatrix} p_\Delta & 0 \\ 0 & p_{inv} \end{bmatrix}$$

Nota: Se exige que $p_\Delta \gg q_\Delta$ y $p_{inv} \gg q_{base}$ para forzar matemáticamente que la trayectoria óptima converja hacia el estado seguro en el infinito, evitando políticas suicidas al borde del horizonte.

7.5.3. 3. El Problema de Optimización (Discretización para el Solver)

El Problema de Control Óptimo (OCP) formateado y compilado para la interfaz C subyacente de CasADi/acados se define como la minimización de la forma cuadrática estandarizada:

$$\min_{u_0 \dots u_{N-1}} \sum_{k=0}^{N-1} (\chi_k^T Q(\Omega) \chi_k + u_k^T R_k(\cdot) u_k) + \chi_N^T P \chi_N$$

sujeto a: $I_{k+1} = I_k + u_{compra,k} - u_{venta,k}$ y a los límites de caja asimétricos dinámicos.

En cada interrupción del Hilo Rápido, el entorno actualiza los parámetros TVP (S_k, Γ_k, Ω , límites de caja), y el sistema llama a `solver.solve()`, resolviendo el problema bajo la garantía de un paisaje de error estrictamente convexo.

7.6. Arquitectura de Concurrencia y Evasión del GIL (Global Interpreter Lock)

Para satisfacer las restricciones de latencia dura del algoritmo de Alta Frecuencia (HFT), el sistema se desacopla en procesos independientes a nivel del Sistema Operativo (OS), evadiendo el cuello de botella del *Global Interpreter Lock* (GIL) de Python. Se emplea una topología híbrida que combina el aislamiento de memoria de `multiprocessing` para la carga matemática, y la concurrencia `asyncio` para la entrada/salida (I/O) de red.

7.6.1. 1. Topología de Procesamiento Aislado

El sistema se divide en tres procesos pesados independientes, cada uno anclado (*pinned*) a un núcleo físico distinto de la CPU para evitar cambios de contexto (*Context Switching*) por parte del planificador del OS:

- **Proceso 1: Motor de Red (Asyncio / I-O Bound):** Dedicado exclusivamente a la conectividad con el *exchange*. Ejecuta un bucle de eventos asíncrono. Su única responsabilidad es mantener abierto el *WebSocket*, recibir los paquetes de datos crudos (P_{spot} , Q_k), estampar el tiempo local (t_{local}) y despachar instantáneamente las órdenes de ejecución dictadas por el optimizador. No realiza ningún cálculo matemático.
- **Proceso 2: Hilo Rápido (EAKF + NMPC / CPU Bound):** Es el núcleo táctico del sistema. Ejecuta la corrección del Filtro de Kalman y el *solver* CasADi/acados. Al estar en un proceso separado, el GIL de este entorno es independiente. Dispara la interrupción a la GPU para resolver la superficie de Loeper y calcula el vector de control óptimo U_0 .
- **Proceso 3: Hilo Lento (Micelio y EMD / CPU Bound):** Responsable del análisis de microestructura. Ejecuta los algoritmos densos iterativos (Descomposición Empírica de Modos y Transformada de Hilbert). Opera de fondo sin interrumpir la latencia del Hilo Rápido, actualizando periódicamente la estabilidad Ω y la frecuencia estructural ω_m .

7.6.2. 2. Comunicación Inter-Procesos (IPC) de Latencia Cero

El paso de mensajes mediante colas (`multiprocessing.Queue`) o tuberías (`Pipes`) requiere serialización y deserialización (*Pickling*) de los objetos de Python, lo cual introduce latencia inaceptable.

Para comunicar los procesos, el sistema emplea **Memoria Compartida a nivel de C** (`multiprocessing.shared_memory` o `ctypes.Array`).

- El **Motor de Red** escribe el último P_{spot} directamente en un bloque de memoria RAM compartida preasignada.
- El **Hilo Rápido** lee este bloque de memoria usando punteros de C (vía NumPy buffers) en $O(1)$ sin pasar por el GIL, garantizando que el dato más fresco esté instantáneamente disponible para el EAKF.
- El **Hilo Lento** escribe los parámetros de riesgo exógeno (Ω, R_n) en un bloque separado. Para evitar condiciones de carrera (*Race Conditions*) durante la escritura matricial, se implementan *Spinlocks* (bloqueos de giro de ultra-baja latencia) en lugar de *Mutexes* tradicionales, minimizando las llamadas al sistema (*syscalls*).
- El **Hilo Rápido** despacha el vector de control táctico (U_0) de vuelta al **Motor de Red** utilizando una estructura de *Ring Buffer* (búfer circular) preasignada en memoria compartida, diseñada bajo un esquema SPSC (*Single-Producer / Single-Consumer*) libre de bloqueos (*Lock-free*). El Motor de Red monitorea esta dirección de memoria mediante *polling* asíncrono continuo; al detectar un cambio de estado en el puntero de escritura, captura instantáneamente la orden (u_{compra} o u_{venta}) y emite el

payload JSON a la API del *broker*, cerrando el lazo de control con latencia sub-milisegundo.

7.6.3. 3. Orquestación del Acelerador Gráfico (GPU)

La llamada a la Ecuación Diferencial Parcial de Loeper se compila en C/CUDA mediante `@cuda.jit` (Numba). Esta llamada se invoca desde el **Hilo Rápido** de manera asíncrona (no bloqueante en la CPU). Mientras los *Streaming Multiprocessors* (SMs) de la GPU construyen la malla $\mathbf{M}_\Gamma$, la CPU del Hilo Rápido adelanta el cálculo de las matrices del EAKF, sincronizándose (`cuda.synchronize()`) únicamente en el milisegundo exacto en que el optimizador NMPC necesita el tensor para inicializar su descenso de gradiente.

7.7. Orquestación Principal y Gestión de Recursos (main.py)

El script principal (`main.py`) actúa como el supervisor jerárquico del sistema. No ejecuta cálculos matemáticos ni operaciones de red; su única responsabilidad es reservar la memoria compartida a nivel del Sistema Operativo, inicializar los procesos de manera segura y gestionar el apagado controlado (Graceful Shutdown) para evitar fugas de memoria y procesos zombis.

7.7.1. 1. Protección del Contexto CUDA (Start Method)

Para garantizar que el Acelerador Gráfico (GPU) asigne los recursos correctamente al Hilo Rápido, el orquestador altera el método de bifurcación predeterminado de Python. Se prohíbe el uso de `fork` (que copia la memoria del proceso padre y corrompe los contextos de los *drivers* de NVIDIA). En su lugar, se impone el método `spawn`:

```
import multiprocessing as mp
if __name__ == '__main__':
    mp.set_start_method('spawn', force=True)
```

7.7.2. 2. Asignación de Memoria Compartida (Shared Memory)

Antes de instanciar los hilos, el supervisor reserva los bloques de memoria RAM estrictamente necesarios, definiendo su tamaño en bytes según las matrices y variables a compartir. Se estructuran tres bloques principales:

- **Bloque de Entorno (Sensor):** Contiene el último precio P_{spot} , la tasa de llegada de ticks ν y el *flag* de *Dropout* manejado por el *Watchdog*.
- **Bloque Estructural (Micelio):** Aloja las variables del Hilo Lento (Ω , ω_m , λ_{sim} y el residuo macro R_n).
- **Bloque de Control (Actuador):** Un *Ring Buffer* SPSC (*Single-Producer / Single-Consumer*) destinado exclusivamente para que el Hilo Rápido escriba el vector de decisión óptimo U_0 .

7.7.3. 3. Inyección de Dependencias e Instanciación

Se inicializan las instancias de `mp.Process`, inyectando los nombres de los bloques de memoria compartida (cadenas de texto únicas) como argumentos. Esto asegura que el Motor de Red, el Hilo Rápido y el Hilo Lento sepan exactamente a qué dirección de memoria conectarse mediante `shared_memory.SharedMemory(name=...)`.

7.7.4. 4. Bucle de Supervisión y Apagado Controlado (Graceful Shutdown)

El orquestador invoca el método `.start()` en los tres procesos y se suspende en un bloque de captura de interrupciones. Es imperativo prevenir fugas de memoria (*Memory Leaks*) en la partición `/dev/shm` del sistema operativo en caso de una terminación abrupta (e.g., `SIGINT` o `SIGTERM`).

La topología de cierre se garantiza mediante una estructura `try...finally`:

1. Se intercepta la orden de cierre.
2. Se envía una señal de terminación (`.terminate()`) a los tres procesos hijos y se espera su cierre confirmando la devolución de recursos (`.join()`).
3. Se invoca la función `.close()` seguida estrictamente de `.unlink()` sobre cada bloque de memoria compartida preasignado, liberando la RAM del servidor y dejando el entorno limpio para la siguiente ejecución.

8. Entorno Real en Cuenta Demo de Binance

Las operaciones de milisegundos no están permitidas, aparte de que la señal se demora físicamente 300 milisegundos para llegar desde Colombia hasta Tokio o Europa, por lo que el enfoque de procesamiento computacional y solicitud de datos ha de cambiar

8.1. Inyección de Fricción Sintética (Liquidez Simulada para Testnet)

Durante la fase de validación en entornos de simulación (Testnet), el libro de órdenes exhibe una liquidez artificialmente infinita y *spreads* nulos. Esto anula el parámetro de impacto λ y evita que la Ecuación Diferencial Parcial de Loeper detecte la Singularidad de auto-liquidación matemática.

Para validar la robustez del controlador predictivo (NMPC) bajo condiciones de estrés real, se desvincula el parámetro de impacto de los datos del *exchange* simulado y se inyecta un tensor de fricción sintética λ_{sim} directamente en la memoria compartida que alimenta a la GPU.

8.1.1. 1. Modelado Estocástico de la Liquidez (Proceso Ornstein-Uhlenbeck)

La falta de liquidez en un mercado real no es ruido blanco; exhibe autocorrelación y reversión a la media (los *market makers* tardan un tiempo finito en reponer la profundidad del libro tras un choque). Para modelar esta memoria térmica, se emplea un proceso de Ornstein-Uhlenbeck discretizado por el método de Euler-Maruyama en el Hilo Lento:

$$\lambda_{ruido,k} = \lambda_{ruido,k-1} + \theta_{OU}(\mu_{OU} - \lambda_{ruido,k-1})\Delta t + \sigma_{OU}\sqrt{\Delta t} \cdot \mathcal{N}(0,1)$$

Parámetros de Calibración:

- μ_{OU} : Nivel medio histórico de falta de liquidez (extraído de métricas reales de Mainnet).
- θ_{OU} : Tasa de reversión a la media (velocidad con la que el *order book* se rellena).
- σ_{OU} : Volatilidad intrínseca del *spread*.
- $\mathcal{N}(0,1)$: Variable aleatoria normal estándar.

8.1.2. 2. Acoplamiento con la Estabilidad Estructural del Micelio

El vacío de liquidez no ocurre en un vacío causal. Físicamente, el impacto de mercado empeora severamente cuando el activo entra en una transición de fase (inestabilidad direccional).

Para correlacionar la fricción estocástica con la dinámica del precio, el parámetro simulado final λ_{sim} se acopla a la derivada de estabilidad Ω :

$$\lambda_{sim} = \max(\lambda_{min}, \lambda_{ruido,k} + \eta \cdot |\Omega|)$$

Donde:

- η : Coeficiente de sensibilidad estructural, con unidades estrictas de $\left[\frac{Ticks^2}{BTC^2}\right]$ para mantener la consistencia dimensional de $[\lambda] = 1/BTC$.
- λ_{min} : Suelo lógico para evitar fricciones negativas computacionales.

8.1.3. 3. Inyección en la Arquitectura Computacional

Durante el modo *Testnet*, el Hilo Lento omite el cálculo de $\lambda = \frac{\omega_m}{\Psi}$ y en su lugar computa λ_{sim} en cada iteración. Este escalar se pasa a través de la memoria compartida hacia la GPU.

Al compilar la malla tridimensional, el núcleo CUDA (`@cuda.jit`) evalúa la función de fricción crítica con el parámetro sintético:

$$\mathcal{D}_{i,j} = 1 - \lambda_{sim} \cdot S_i^2 \cdot \Gamma_{i,j}$$

Consecuencia Operativa: Si el bot ejecuta operaciones sobredimensionadas en momentos donde el ruido de Ornstein-Uhlenbeck está en un pico, o cuando la estabilidad Ω se corrompe, $\mathcal{D}_{i,j}$ colapsará hacia cero de forma realista. El *solver* CasADi detectará la singularidad de Loeper en su descenso de gradiente y activará el freno de emergencia, demostrando empíricamente que la mitigación de riesgo funciona sin arriesgar capital real.

8.2. Mitigación de Saturación de Red (Token Bucket Asíncrono)

La API de Binance (tanto en Testnet como en Mainnet) no penaliza por cantidad bruta de solicitudes, sino por un sistema de pesos acumulativos (Weight Limits) por minuto y por segundo. Si el bot, en un escenario de alta volatilidad ($\Omega \gg 0$), dispara múltiples órdenes o cancelaciones simultáneas, agotará el crédito de la IP, resultando en un error HTTP 429 y un baneo temporal.

Para garantizar la operabilidad continua sin exceder los límites del *exchange*, el Motor de Red implementa un limitador de tasa basado en el algoritmo *Token Bucket* (Cubo de Fichas), ejecutado como un *middleware* asíncrono.

8.2.1. 1. Definición Matemática del Token Bucket

El sistema mantiene un depósito virtual de tokens que representa el límite de peso permitido por la API de Binance. El estado del cubo en un instante discreto k se define por la ecuación de conservación:

$$T_k = \min(C_{max}, T_{k-1} + \rho \cdot \Delta t) - w_k$$

Donde:

- T_k : Fichas disponibles en el instante actual.
- C_{max} : Capacidad máxima del cubo (límite de peso por intervalo estipulado por el *exchange*).
- ρ : Tasa de regeneración continua de fichas ($\frac{C_{max}}{60}$ para límites por minuto).
- Δt : Tiempo físico transcurrido en segundos desde la última actualización del *bucket*.
- w_k : Peso específico de la solicitud entrante (e.g., Orden GTC = 1, Cancelación = 1, *Polling* de cuenta = 10).

8.2.2. 2. Arquitectura de Despacho (Middleware)

Antes de que el Motor de Red emita cualquier solicitud (HTTP POST/GET o WebSocket frame), la petición debe atravesar la compuerta de validación del *Token Bucket*:

1. **Evaluación de Disponibilidad:** Se actualiza T_k usando el reloj del sistema. Si $T_k \geq w_k$, la compuerta se abre de inmediato, la solicitud se despacha, y se descuenta el peso w_k del cubo.

2. **Gestión de Agotamiento (Encolamiento vs. Descarte):** Si $T_k < w_k$, el límite está saturado. El sistema clasifica la petición mediante su etiqueta de prioridad:

- *Baja Prioridad (Sondeo o Sincronización):* Se descarta inmediatamente para no asfixiar la cola de red.
- *Alta Prioridad (Ejecución de U_0 o Cancelaciones de Emergencia):* Se inyecta en una cola asíncrona (`asyncio.PriorityQueue`). El hilo `await` entra en modo de suspensión no bloqueante calculando el retardo exacto para la disponibilidad: $\tau_{espera} = \frac{w_k - T_k}{\rho}$. Una vez cumplido τ_{espera} , la petición se ejecuta.

8.2.3. 3. Fallback Exponencial frente a Errores de Servidor

Incluso con el *Token Bucket* activo, la infraestructura del *exchange* puede responder con códigos 429 (sobrecarga del motor de emparejamiento, independiente del usuario) o errores 5xx.

Para evitar un ciclo de retroalimentación destructivo donde el bot reinteente agresivamente despachar una orden fallida, el manejador de la cola aplica un Decaimiento Exponencial (*Exponential Backoff*):

$$\tau_{reintento,n} = \tau_{base} \cdot 2^n + \mathcal{U}(0, j_{max})$$

Donde n es el número del reintento, τ_{base} es el tiempo inicial, y $\mathcal{U}(0, j_{max})$ es un factor de *Jitter* (ruido uniforme aleatorio) para evitar la sincronización de colisiones de red. Si $n > 3$, la orden se purga de la cola para evitar que el bot ejecute operaciones basadas en un estado obsoleto del mercado.

8.3. Degradación Inercial y Preservación del Grafo de Optimización

Cuando el limitador de tasa (*Token Bucket*) bloquea temporalmente las peticiones de red hacia la API de Binance para prevenir sanciones por sobrecarga, el sistema entra en un estado de ceguera temporal. Durante este intervalo, la arquitectura de control no debe intentar compensar el tiempo inactivo con cálculos más densos dentro del optimizador, sino transicionar a un modelo de inercia puramente cinemática.

8.3.1. 1. Crecimiento Progresivo de la Incertidumbre (Matriz P)

Al no recibir actualizaciones del *exchange* ($z_k = \emptyset$), el Filtro de Kalman Adaptativo (EAKF) suspende su fase de corrección. La Ganancia de Kalman se anula ($K_k = 0$), y la estimación del estado se rige estrictamente por la matriz de transición A :

$$x_{k|k} = Ax_{k-1|k-1}$$

Consecuentemente, la matriz de covarianza del error P no se contrae. En su lugar, absorbe iterativamente la matriz de ruido del proceso Q_k , provocando que la incertidumbre total (medida por su traza) crezca de manera simétrica y progresiva:

$$P_{k|k} = AP_{k-1|k-1}A^T + Q_k$$

Este aumento de P se transmite directamente a la Ecuación Diferencial Parcial (EDP) de Loeper a través del término de varianza instantánea σ^2 , lo que ensancha preventivamente el cono de riesgo a medida que el sistema opera a ciegas.

8.3.2. 2. Restricción de Autodiferenciación (El Peligro de Acoplar Loeper)

Resulta imperativo documentar una restricción computacional estricta: la EDP estocástica de Minimax-Loeper **no debe, bajo ninguna circunstancia, integrarse dentro del bucle de predicción interna del solver NMPC**, independientemente del tiempo ocioso de CPU generado por la latencia de red.

El optimizador CasADi/acados emplea Programación Cuadrática Secuencial (SQP), requiriendo el cálculo exacto de Jacobianos y matrices Hessianas mediante Diferenciación Algorítmica (AD) sobre un grafo simbólico precompilado en C. Si la resolución de la EDP se incrusta como una restricción dinámica por cada nodo de predicción del horizonte N , se desencadenan dos colapsos sistémicos:

- **Pérdida de Dispersión (*Sparsity*):** La derivación a través del integrador numérico de Loeper densifica las matrices del problema, destruyendo la convexidad y provocando que la complejidad computacional diverja hacia $\mathcal{O}(N^3)$, congelando el hilo de control.
- **Violación de Memoria (GPU-CPU):** El grafo simbólico de CasADi no puede rastrear derivadas a través de funciones ejecutadas de forma asíncrona en los núcleos CUDA (`@cuda.jit`). Esto induciría fallos de segmentación (*segmentation faults*) a nivel del sistema operativo.

8.3.3. 3. Resolución Asíncrona (Interpolación $\mathcal{O}(1)$)

Para preservar la estabilidad algorítmica, el modelo físico y el modelo de control se mantienen matemáticamente desacoplados:

1. La GPU compila y actualiza la malla tridimensional estática $\mathbf{M}_\Gamma$ únicamente cuando el EAKF procesa una *nueva* medición válida. Durante las pausas de red del *Token Bucket*, la malla permanece congelada en su último estado válido.
2. Durante su descenso de gradiente iterativo, el controlador NMPC lee los valores de convexidad $\Gamma(S_k, t_k)$ y riesgo asumiendo la malla de la GPU

como un Parámetro Variable en el Tiempo (TVP). Utiliza *B-splines* para interpolar la superficie en tiempo $\mathcal{O}(1)$.

Esta arquitectura garantiza que, frente a estrangulamientos de la API, el bot asuma una postura defensiva impulsada por el crecimiento de P , sin destruir los gradientes del optimizador ni violar los límites de latencia del *hardware*.

8.4. Resiliencia del Enlace Asíncrono y Manejo de Tramas de Control

Para mantener una automatización algorítmica ininterrumpida y de alta frecuencia, es imperativo sortear las políticas de desconexión por inactividad de los *exchanges*. Plataformas como Binance emiten tramas de control (*Ping frames*) a través del WebSocket, exigiendo una respuesta (*Pong frame*) en un límite de tiempo estricto. Si el *Event Loop* de la aplicación se encuentra bloqueado, el servidor asume una caída del cliente y corta la conexión.

Para garantizar la perpetuidad del enlace sin comprometer la latencia de ejecución, el sistema implementa un diseño de aislamiento concurrente y auto-recuperación:

8.4.1. 1. Desacople I/O y Delegación de Tramas

El bucle de eventos (`asyncio Event Loop`) reside exclusivamente en el Proceso 1 (Motor de Red). Se prohíbe estrictamente la ejecución de cualquier operación CPU-Bound (como cálculos matriciales, interpolaciones o descensos de gradiente) dentro de este entorno.

Al garantizar que el Proceso 1 sea 100 % asíncrono y no bloqueante, las librerías de red subyacentes interceptan y responden automáticamente a las tramas *Ping* del servidor en el *background*, haciendo que el mantenimiento de la conexión sea invisible para la lógica táctica del bot.

8.4.2. 2. Arquitectura de Reconexión Automática (Watchdog)

Dado que las desconexiones a nivel de protocolo (TCP/IP) son eventualidades estocásticas ineludibles en la infraestructura global de internet, el Motor de Red se encapsula dentro de una corrutina de supervisión o *Watchdog*.

La lógica matemática de reconexión infinita se estructura como:

- **Detección de Ruptura:** Si el *socket* arroja una excepción de cierre (`ConnectionClosed`) o se agota el tiempo de espera de lectura, el *Watchdog* intercepta el error antes de que se propague al sistema operativo.
- **Señalización de Ceguera:** Instantáneamente, el Proceso 1 escribe un indicador booleano (`FLAG_DROPOUT = True`) en la memoria compartida. Esto notifica al Proceso 2 (Hilo Rápido) para que anule la Ganancia de

Kalman ($K = 0$) y permita que la incertidumbre de la matriz P absorba el vacío de datos.

- **Reconexión sin Bloqueo:** El *Watchdog* reestablece la conexión de red utilizando un decaimiento exponencial para los intentos iterativos, reanudando la ingesta de P_{spot} sin necesidad de reiniciar el software ni corromper el histórico de las variables de estado del Filtro de Kalman.

8.5. Cuantización del Vector de Control (Filtros de Exchange)

El optimizador predictivo (NMPC) opera en el dominio de los números reales continuos ($\mathbb{R}$). Consecuentemente, el vector de decisión táctico U_0 generado poseerá precisión de punto flotante infinita (e.g., $u_{compra} = 0,1458923$ BTC).

Sin embargo, los motores de emparejamiento de los *exchanges* (como Binance) operan estrictamente en un dominio discreto, imponiendo filtros de normalización absolutos. Si una orden viola estos filtros, es rechazada instantáneamente por el servidor arrojando un error de sintaxis comercial.

Para garantizar la ejecución exitosa, el Motor de Red debe aplicar un paso de cuantización geométrica a U_0 extrayendo previamente las reglas del símbolo desde el *endpoint* `exchangeInfo`. Se aplican dos filtros secuenciales:

1. **Filtro de Tamaño Mínimo (MIN_QTY y MIN_NOTIONAL):** La orden de volumen u se evalúa contra el tamaño mínimo permitido por la plataforma y el valor notional mínimo exigido en USD. Si el cálculo óptimo del NMPC exige una operación microscópica que no cumple la condición empírica, la orden se fuerza analíticamente a cero:

$$u_{eval} = \begin{cases} u, & \text{si } (u \geq \text{minQty}) \wedge (u \cdot P_{spot} \geq \text{minNotional}) \\ 0, & \text{en caso contrario} \end{cases}$$

2. **Filtro de Discretización Escalonada (LOT_SIZE):** El volumen de la orden no puede tener más decimales que los permitidos por la regla de incremento (`stepSize`). Se debe aplicar una función de piso matemático (*floor*), no un redondeo estándar (*round*), para garantizar que jamás se intente comprar una fracción minúscula que exceda el capital líquido disponible:

$$u_{ejecucion} = \left\lfloor \frac{u_{eval}}{\text{stepSize}} \right\rfloor \cdot \text{stepSize}$$

Solo después de someter el vector a esta discretización, el escalar resultante $u_{ejecucion}$ es serializado e inyectado en el *payload* de la API. Esta corrección algorítmica asegura un índice de aceptación del 100% en las órdenes tácticas emitidas, manteniendo la cohesión entre la teoría de control y la topología del mercado.

8.6. Telemetría Asíncrona y Registro Analítico (Data Logging)

Ningún modelo estocástico nace perfectamente calibrado. Para permitir el ajuste fino de la matriz de ruido del proceso Q y la validación de la matriz de transición A tras periodos de prueba prolongados (e.g., 15 días), el sistema debe almacenar el comportamiento histórico de las variables de aumento de estado y los errores de estimación.

Acumular estas series de tiempo en la memoria RAM principal (arrays dinámicos) introduciría una degradación severa de latencia y un colapso inminente por desbordamiento de memoria (OOM). Por ello, se implementa un mecanismo de volcado a disco en lotes (*Chunked Flushing*).

8.6.1. 1. Variables Objetivo de Calibración

El sistema de telemetría no guarda todos los cálculos intermedios, sino que aísla estrictamente los vectores necesarios para aplicar algoritmos de *Covariance Matching* en diferido (offline):

- **Vector de Estado (x_k):** Específicamente el comportamiento de las variables de aumento (como el residuo macro R_n o cualquier sesgo b introducido) para verificar que convergen a constantes físicas reales y no oscilan ciegamente.
- **La Innovación ($\tilde{y}_k$):** Es el error puro del modelo. Si el Filtro de Kalman está bien calibrado, la serie de tiempo de la innovación debe comportarse como ruido blanco gaussiano de media cero. Cualquier autocorrelación detectada aquí indica que la matriz A tiene defectos dinámicos.
- **Incertidumbre ($Tr(P)$):** Para mapear cómo el sistema reaccionó a los bloqueos de red o vacíos de liquidez.

8.6.2. 2. Arquitectura de Volcado (Parquet/HDF5)

El Hilo Rápido preasigna un *buffer* estático en memoria NumPy de tamaño fijo (e.g., $N = 10,000$ iteraciones). En cada ciclo, el filtro de Kalman sobrescribe el índice correspondiente en $O(1)$. Cuando el *buffer* se llena, el Hilo Rápido no detiene su ejecución para guardar el archivo; en su lugar, delega una copia superficial (o emite una señal a un hilo secundario de I/O) para que este escriba los datos en el disco duro de estado sólido (NVMe) en formato **.parquet** o **.h5**.

Esta separación garantiza que el análisis forense del comportamiento del bot posea datos de resolución ultra-alta (nivel *tick*) sin entorpecer el bucle de optimización táctica del NMPC.